Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет Программной инженерии и компьютерной техники

Образовательная программа СППО

Направление подготовки (специальность) 09.03.04

О Т Ч Е Т

об производственной, технологической практике

Тема задания: Разработка, настройка workflow-движка и логики кеширования в ECM-системе

Обучающийся Дениченко Александр Олегович*, P3412*

Руководитель практики от университета: *Маркина Татьяна Анатольевна, старший преподаватель*

Руководитель практики от компании: *Филимонов Антон Борисович, руководитель центра компетенций по разработке прикладных решений в ООО “Газпром ЦПС”*

Дата 16.02.26

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 ОСНОВНАЯ ЧАСТЬ **9**

1.1 Утилита для сбора логов по camunda events 9

1.1.1 Задача 9

1.1.2 Аналитика задачи 9

1.1.3 Заведение задачи в системе трекинга 10

1.1.4 Исследование текущего интерфейса системы бизнес процессов 10

1.1.5 Контроллер 12

1.1.6 Сервис 13

1.1.7 Затронутое API 14

1.1.8 Допустимые ошибки 15

1.1.9 Замер потребления ресурсов при нагрузке 15

1.1.10 Использованные технологии 17

1.1.11 Почему были выбраны технологии 17

1.1.12 Вывод 18

1.2 Дописывание получения движений токена по workflow 18

1.2.1 Задача 18

1.2.2 Аналитика задачи 18

1.2.3 Исследование точек внедрения 19

1.2.4 Handler 20

1.2.5 Сервисы для обработки событий на уровне плагина 21

1.2.6 Затронутое API 24

1.2.7 Дописывание API 25

1.2.8 Результаты сообщений из брокера 26

1.2.9 Проверка корректного восстановления 27

1.2.10 Вывод 30

1.3 Настройка движка Camunda под нагрузку 30

1.3.1 Задача 30

1.3.2 Аналитика задачи 30

1.3.3 Написание конфигурации Job Executor 31

1.3.4 Настройка параметров длительного опроса 32

1.3.5 Отключение уровня хранения истории 32
1.3.6 Конфигурационные файлы 32
1.3.7 Проверка быстрой асинхронной работы job executors 33
1.3.8 Проверка отсутствия истории в слое персистенции при движении по схеме 33
1.3.9 Вывод 33
1.4 Написание кеша для метаданных 33
1.4.1 Задача 33
1.4.2 Аналитика задачи 34
1.4.3 Реализация решения 34
1.4.4 Модель данных 35
1.4.5 Слой получения данных 35
1.4.6 Парсинг иерархии справочников 35
1.4.7 Кеш и инициализация 36
1.4.8 Асинхронная инициализация и повторные попытки 36
1.4.9 Конфигурация 37
1.4.10 Тестирование 37
1.4.11 Интеграция решения 37
1.4.12 Заполенения кеша 38
1.4.13 Чтения из кеша 38
1.4.14 Инвалидации кеша 39
1.4.15 Снижения обращений в базу 39
1.4.16 Вывод 39

ЗАКЛЮЧЕНИЕ 40

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 41

ПРИЛОЖЕНИЕ А (Система сбора истории камунды) 42

ПРИЛОЖЕНИЕ Б (Дописывание получения движений токена по workflow) 49

ПРИЛОЖЕНИЕ В (Конфигурация движка Camunda под нагрузку) 66

ПРИЛОЖЕНИЕ Г (Написание кеша для метаданных) 68

ВВЕДЕНИЕ

Производственная практика является обязательным этапом образовательного процесса, который направлен на проверку компетенцией обучающегося. Данная проверка подразумевает по собой проверку компетенций, полученных в процессе освоения образовательной программы направления – системное прикладное и программное обеспечение, путём работы над настоящими задачами в настоящем проекте, где трудовой распорядок максимально приближен к официальной занятости посредством трудоустройства.

Данный отчёт отражает работу в рамках практики.

Цель практики заключалась в демонстрации учащимися приобретённых навыков работы по специальности.

Чтобы выполнить поставленную цель были проделаны следующие этапы, приведённые в таблице 0.1.

Таблица 0.1 – Этапы и задания практики

Название этапа	**Задание**
Инструктаж обучающегося	Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка
Утилита для сбора логов по camunda events	- Сервис реализует API для получения истории событий Camunda за произвольный временной диапазон с параметрами startTime и endTime, передаваемыми через URI. - Возвращаемые данные включают только активности типов: service task, user task, exclusive gateway, send task, start event, script task, inclusive gateway, intermediate message throw event. - Формат выгрузки данных — .xlsx, со столбцами Activity ID, Activity Name, Start Time, End Time, Duration (ms), Activity Type. - Сбор истории выполняется без заметного влияния на производительность основной системы (разница в средней загрузке CPU не более 5%). - Ошибки при запросе возвращают корректный HTTP-ответ с кодами: 400 (некорректные параметры), 500 (внутренняя ошибка). В отчёт войдёт: описание шагов для аналитики решения; описание шагов решения; само решение в коде (контроллер, сервис, возможно затронутое api и его дописывание); файлы, которые будут формироваться при запросе; обработка ошибок; замер метрик цпу при нагрузке на камунду для сверки средней нагрузки (проверка интрузивности); вывод.
Дописывание получения движений токена по workflow	- Получение истории и статуса токенов переведено на асинхронную схему: Camunda самостоятельно публикует уведомления о событиях в брокер сообщений. - На стороне бизнес-логики реализован модуль подписки на эти уведомления без периодического опроса (polling). - Подключение брокера выполнено в виде плагина Camunda Engine, активируемого при старте standalone-сборки. - Система корректно восстанавливает состояние подписки после перезапуска.

Продолжение таблицы 0.1

Название этапа	**Задание**
	В отчёт войдёт: описание шагов для аналитики решения; описание шагов решения; само решение в коде (хендлер, сервис/ы, затронутое api, его дописывание и код плагина); результаты сообщений из брокера; проверка корректного восстановления и перечитывания пропущенных событий из брокера; вывод.
Настройка движка Camunda под нагрузку	- Настроен встроенный брокер Camunda для эффективной работы при высокой нагрузке с external tasks. - Job Executors оптимизированы под асинхронное выполнение задач с минимальным временем ожидания. - Запись истории в слой персистенции отключена; при этом логика работы брокера полностью сохраняется. В отчёт войдёт: описание шагов для аналитики решения; описание шагов решения; само решение (конфигурационные файлы); проверка быстрой асинхронной работы job executors; проверка отсутствия истории в слое персистенции при движении по схеме; проверка, что это решение не затронуло брокер; вывод.
Написание кеша для метаданных	- Реализован read model кеш, наполняемый при старте сервиса из слоя персистенции. - Кеш подключается как отдельная библиотека с минимальным набором зависимостей. - Инвалидация кеша осуществляется по событиям через event listener без перезапуска сервиса. - При обращении к данным из кеша количество обращений к базе данных происходит только на заполнение кеша. - Кеш корректно обновляется при получении события изменения метаданных. В отчёт войдёт: описание шагов для аналитики решения; описание шагов решения; само решение кодом (сервисы библиотеки, зависимости, интеграция библиотеки); проверка заполенения кеша; проверка чтения из кеша; проверка инвалидации кеша; проверка снижения обращений в базу (cache hit rate); проверка обновлений кеша; вывод.

Продолжение таблицы 0.1

Название этапа	**Задание**
Оформление отчётных документов в соответствии с требованиями	1. Должно быть подробное описание выполнения задач по этапам. Результаты задания необходимо разместить в приложения. 2. Оформление отчёта должно быть выполнено в соответствии с методическим пособием (https://books.ifmo.ru/file/pdf/2622.pdf) 3. Структура документа: титульный лист, введение, основная часть, заключение, приложения. 4. В основной части подробно описывается выполнение задач 2-5 этапов, в приложении помещаются результаты данных этапов. 5. Отчёт необходимо подгрузить в модуле практика как ”письменный отчёт”.
Получение отзыва руководителя практики	Получить отзыв с оценкой у руководителя практики в модуле Практики.
Техническое задание по ВКР	Написать техническое задание по ВКР. Техническое задание должно содержать: наименование, назначение, основание для разработки, функции, структура, пользовательский интерфейс, требования к надёжности, требования к безопасности, условия эксплуатации и проч. важные требования, документация, стадии и этапы разработки, порядок контроля и приёмка. В случае отсутствия какого-либо пункта необходимо обосновать его отсутствие. Сформировать в виде документа в формате pdf и прислать на проверку на электронную почту преподавателя.
Оформление отчётных документов в соответствии с требованиями	1. Должно быть подробное описание выполнения задач по этапам. Результаты задания необходимо разместить в приложения. 2. Оформление отчёта должно быть выполнено в соответствии с методическим пособием (https://books.ifmo.ru/file/pdf/2622.pdf). 3. Структура документа: титульный лист, введение, основная часть, заключение, приложения. 4. В основной части подробно описывается выполнение задач 2–4 этапов, в приложении помещаются результаты данных

Продолжение таблицы 0.1

Название этапа	**Задание**
	этапов. 5. Отчёт необходимо подгрузить в модуле практика как «письменный отчёт».
Получение отзыва руководителя практики	Получить отзыв с оценкой у руководителя практики в модуле Практики

1 ОСНОВНАЯ ЧАСТЬ

1.1 Утилита для сбора логов по camunda events

1.1.1 Задача

- Сервис реализует API для получения истории событий Camunda за произвольный временной диапазон с параметрами startTime и endTime, передаваемыми через URI.
- Возвращаемые данные включают только активности типов: service task, user task, exclusive gateway, send task, start event, script task, inclusive gateway, intermediate message throw event.
- Формат выгрузки данных — .xlsx, со столбцами Activity ID, Activity Name, Start Time, End Time, Duration (ms), Activity Type.
- Сбор истории выполняется без заметного влияния на производительность основной системы (разница в средней загрузке CPU не более 5%).
- Ошибки при запросе возвращают корректный HTTP-ответ с кодами: 400 (некорректные параметры), 500 (внутренняя ошибка).

1.1.2 Аналитика задачи

Задача была получена и начата на одном из обсуждений текущих приоритетов команды 16.02.2026. Даннуя задачу начали обсуждать в предверии нагрузочного тестирования всей системы, для сверки метрик, которые были поставлены в документации проекта. Так как аудит должен был включать проверку каждого сервиса системы, то требовалось проверить, не является ли система бизнес процессов узким местом системы. После получения задачи, я связался с аналитиком, который мне более детально объяснил, что требуется от механизма сбора данных, а если конкретнее, то нужно было собрать все активности для отслеживания токена процесса следующего вида: service task, user task, exclusive gateway, send task, start event, script task, inclusive gateway, intermediate message throw event, так же требовалось дополнительная информация о номере процесса, его имени, времени начала, времени конца, затраченного времени на выполенения конкретной активности и её тип. Получе-

ние этой информации аналитик попросил реализовать интерфейс ввиде простых запросов по протоколу передачи гипертекста. Для получения данных был предложен следующий формат запроса:

Listing 1: Запрос лога активности

```
exmaple.ru/api/activity-log?startTime=2026-06-11T11:00:00&endTime
   =2026-06-11T14:28:00
```

Данный запрос включает в себя параметры, который решено передавать через URI. Параметры включают: время с которого стоит делать сбор активностей и время окончания сбора данных. Указание временного промежутка должно помочь аналитикам при работе с несколькими процессами, так как кол-во строк истории часто измеряется в сотнях тысяч. Так же это позволит делать срезы, наиболее интересные для исследования в точках зависания, чтобы просмотреть, какие конкретно активности исполнялись. Формат для представления данных был выбран ввиде exel-отчёта.

1.1.3 Заведение задачи в системе трекинга

После перевода задачи аналитики в статус ”Выполнена”. Я ещё раз ознакомился с аналитикой и создал задачу для разработки, прикрепил в ”Относится к” задачу аналитики. Проставил срок для выполнения с 16.02.2026 по 05.03.2026, в этот срок так же закладываю время на тестирование нового механизма. Задаче были назначены следующие метки – бизнес-логика, задача и проставлена веха, в которую попадёт задача. Критичность выставлена ”Средний”, так как задача не блокирует другие задачи и не является багом. После описания задачи, я перевёл её в статус ”В процессе”.

1.1.4 Исследование текущего интерфейса системы бизнес процессов

Для реализации задачи нужно изначально изучить интерфейс, который предоставляет нам система бизнес процессов для запроса истории. На официальном сайте с описанием интерфейсов [1] я нашёл нужный мне метод для сбора истории. Данный метод поддерживает фильтрацию по временному промежутку с помощью параметров запроса, что полностью соответствует требованию аналитика о возможности сбора активностей за определённый временной интервал. Кроме того, метод возвращает следующие поля для

каждой активности:

1. id — уникальный идентификатор экземпляра активности;
2. activityId — идентификатор активности;
3. activityName — имя активности;
4. activityType — тип активности;
5. processDefinitionId — идентификатор определения процесса;
6. processInstanceId — номер экземпляра процесса;
7. startTime — время начала выполнения активности;
8. endTime — время окончания выполнения активности;
9. duration — затраченное время в миллисекундах.

Данные поля нам подходят по бизнес требованиям. Так как мы используем клиентскую библиотеку для взаимодействия с отдельными модулем камунды, то имеет смысл воспользоваться инверсией контроля. Сам сервис мы будет использовать при помощи внедрения зависимостей через конструктор при помощи аннотирования класса для снижения прогретого кода. У клиента есть несколько методов, которые подходят нам, но я выбираю метод, где на вход возможны 3 параметра: номер страницы для пагинации, размер страницы и объект с параметрами фильтрации запроса. Данный метод позволяет нам сразу передать в теле запроса объект с фильтрами, включая параметры $startedAfter$ и $startedBefore$. Возвращает он список объектов, содержащих все необходимые поля: идентификатор активности, её тип, время начала и окончания, длительность выполнения в миллисекундах, а также идентификаторы экземпляра процесса и определения процесса. Всё это даёт возможность собирать активности только за указанный аналитиком период времени, что было одним из ключевых требований задачи. Выбранный метод использует POST-запрос. Параметры временного диапазона передаются в теле запроса, а не в URL.

1.1.5 Контроллер

Реализацию задачи я решил начать сверху вниз, начиная с интерфейса у бизнес логики. На проекте используются стандартные для фреймворка контроллеры, запрос в которые приходит из специального сервлета, который находит конкретный конроллер и метод по заданному пути. Разработка контроллера началась с определения точки входа. Сам тип запроса выбран как GET, так как операция не изменяет данные на уровне сервиса, а только возвращает их, это соответствует семантике типа метода. Для поддержки временной фильтрации в контроллер были добавлены необязательные параметры: время начала и время конца. Они принимаются в формате в стандартном ISO-формате, как это было заданно в аналитической задаче. Оба параметра помечены как необязательные, чтобы при необходимости можно было выгрузить полный журнал без временных ограничений. Добавил валидацию для ошибочных запросов, которая учитывает:

1. начальное время не может быть позже конечного;
2. начальное время не может быть в будущем;
3. конечное время не может быть в будущем.

При нарушении любого из этих условий будет ответ с кодом 400, сообщая клиенту о неверно сформированном запросе. В контроллер добавил логирование для отслеживания факта обращения к эндпоинту, а также логирование ошибок и предупреждений. Это позволяет в дальнейшем анализировать работу механизма. Контроллер не занимается непосредственно сбором данных, поэтому я решил делегировать эту работу сервису, вызывая его метод, так как это соответствует принципу разделения ответственности. После получения файла от сервиса контроллер проверяет, не равен ли результат пустому значению. Если данных за указанный период не найдено, возвращается ответ с кодом 404. Если файл успешно создан, контроллер формирует имя файла с текущей датой и временем, чтобы каждый скачанный файл имел уникальное имя и не перезаписывал предыдущие. Для возврата файла контроллер использует потоковый режим, который позволяет не загружать его полностью в память. Это важно при работе с большими объёмами данных. В заголовки ответа добавляется тип для Excel-файлов. Код ответа — 200. В случае воз-

никновения ошибки при генерации файла (например, проблем с доступом к Camunda) контроллер возвращает ответ с кодом 500, логируя при этом детали ошибки для последующего анализа. По требованиям ревью, я сделал документацию контроллеру с помощью аннотаций Swagger. В описании указаны возможные коды ответов, форматы параметров и описание работы метода. Сам код контроллера прикреплён в **Приложении А**.

1.1.6 Сервис

Для реализации самой выгрузки данных и формировании отчёта, был разработан сервис. Его основное назначение – собрать данные об активностях за заданный промежуток времени и сформировать exel-файл с этими данными. Так же дополнительно из конфигурации были вытянуты человекочитаемые совпадения из бизнес процесса для сопоставления с уникальными названиями активностей. При запуске сервиса, модуль подгружает человекочитаемые названия из файла, который лежит в classpath. Загрузка выполняется однократно при помощи введения некоторой аннотации $PostConstruct$. Само местоположение файла можно задавать через конфигурационный параметр для гибкости использования. Предзагрузка сделана для создания закешированного вида файла, чтобы ускорить работу с сопоставлением имён. Для получения исторических записей, я использовал клиентскую библиотеку, внедрение зависимостеи я сделал посредством аннотации $RequiredArgsConstructor$. Я сделал создание транспортного объекта для запроса, куда включил фильтры, которые были переданы: $startedAfter$ и $startedBefore$. Если параметры не переданы, фильтры не применяются, и выгружаются все доступные активности. Из полученного от Camunda списка активностей отфильтровываются только те, у которых заполнено поле $endTime$. Я сделал такую фильтрацию так как незавершённые активности не имеют полной информации о времени выполнения и длительности, что делает их бесполезными для анализа. Так же я предусмотрел излишнюю работу и если список пустой, то пользователю вернётся информативный ответ с ошибкой, что за заданный промежуток ничего не найдено. Для формирования выходного файла я использовал библиотекту $ApachePOI$, которая позволила мне генерировать файлы нужного формата. Сервис создаёт новую книгу, добаляет туда лист, формирует строку заголов-

ков и далее заполняет таблицу. При заполнении каждой строки я выполняю некоторые преобразования:

1. Даты startTime и endTime форматируются в человекочитаемый вид.

2. Технические идентификаторы активностей преобразуются в понятные названия, который обращается к загруженному при старте справочнику.

3. Для родительских активностей также извлекаются их названия и схемы.

При формировании файла они сохраняются во временную директорию, это я сделал чтобы не оставлять файлы на диске после завершения запроса пользователя. При возникновении ошибок сервис должен вывести тело ошибки и выбрасывать исключение, которое далее обработается на уровне контроллера. После записи файла книга закрывается, а поток вывода завершается с помощью конструкции $try-with-resources$, что так же будет гарантировать освобождение системных ресурсов.

Файл с человекочитаемыми названиями имеет следующую структуру:

Listing 2: Файл с человекочитаемыми названиями

```
{
  "ServiceTask_1": {
    "name": Проверка" данных",
    "schema": Проверка" входных параметров"
  },
  "UserTask_1": {
    "name": Согласование"",
    "schema": Утверждение" заявки руководителем"
  }
}
```

Сам код сервиса можно увидет в **Приложении А**.

1.1.7 Затронутое API

В ходе реализации я задействовал следующие компоненты REST API Camunda:

– POST $/history/activity - instance$ — метод, выполняющий запрос исторических записей активностей. В теле запроса я передаю параметры фильтрации: $startedAfter$ и $startedBefore$.

– Клиентская библиотека Camunda 7 Java Client — предоставляет обёртку $HistoricActivityInstanceApi$, которая инкапсулирует формирование HTTP-запроса и десериализацию ответа.

1.1.8 Допустимые ошибки

В процессе сбора логов возможны некоторые ошибки, обработку которых я предусморел и представил в Таблице 1.1.

Таблица 1.1 – Коды ответов и их значения

Код	**Ситуация**
400	$startTime$ позже $endTime$; любая из дат находится в будущем; неверный формат даты.
404	За указанный период не найдено ни одной завершённой активности (список пуст после фильтрации).
500	Ошибка подключения к Camunda; ошибка чтения JSON-файла с маппингом имён; ошибка записи временного Excel-файла; необработанное исключение API.

1.1.9 Замер потребления ресурсов при нагрузке

Замер я проводил на специальном контуре, где были созданы процессы. Один из процессов я запустил вручную, чтобы начала щаписываться история. Сам процесс был запущен в 22:49 (время специфично для сервера и не совпадает с реальным $GMT + 3$). После завершением процесса в 22:53, я сделал запрос на сбор истории в 22:54.

Listing 3: Запрос лога активности

```
gpz.prod.ru/api/activity-log?startTime=2026-02-22T15:00:00&
   endTime=2026-02-27T18:00:00
```

Для рассмотрения нагрузки я воспользовался готовым инструментом сбора метрик по $CPUUsage$ и $MemoryUsage$.

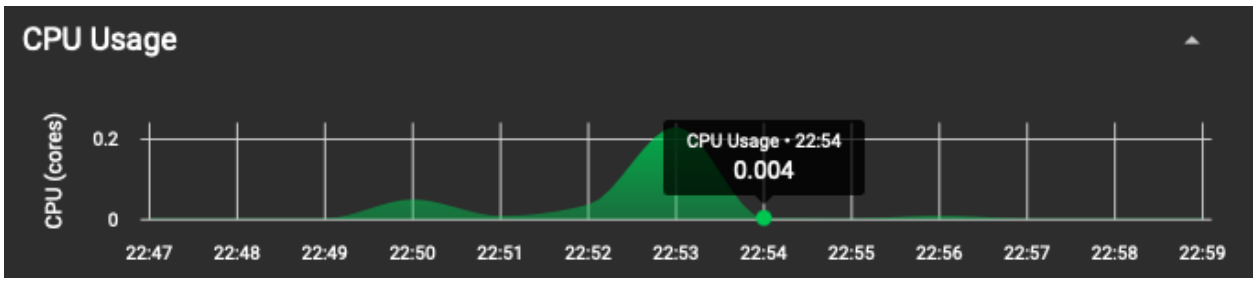

Рисунок 1.1 – Скриншот графика загрузки процессора сервиса со сбором активностей

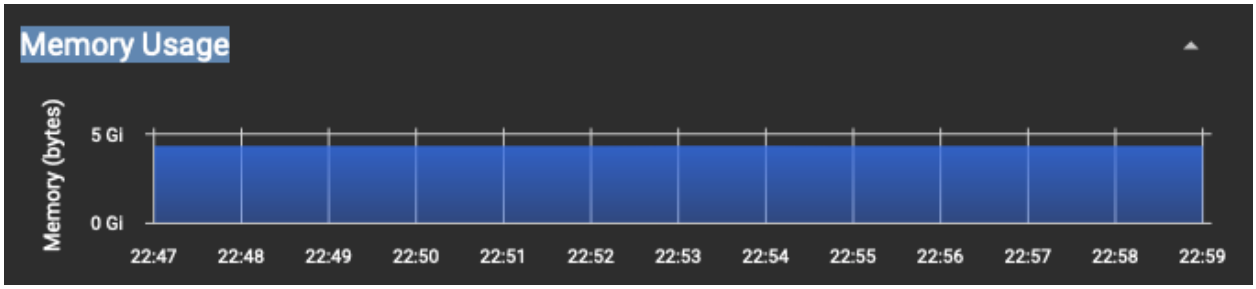

Рисунок 1.2 – Скриншот графика загрузки памяти сервиса со сбором активностей

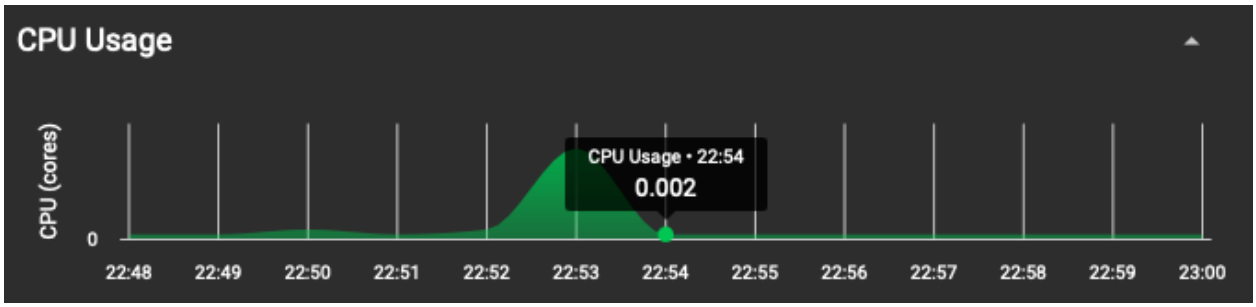

Рисунок 1.3 – Скриншот графика загрузки процессора сервиса с camunda

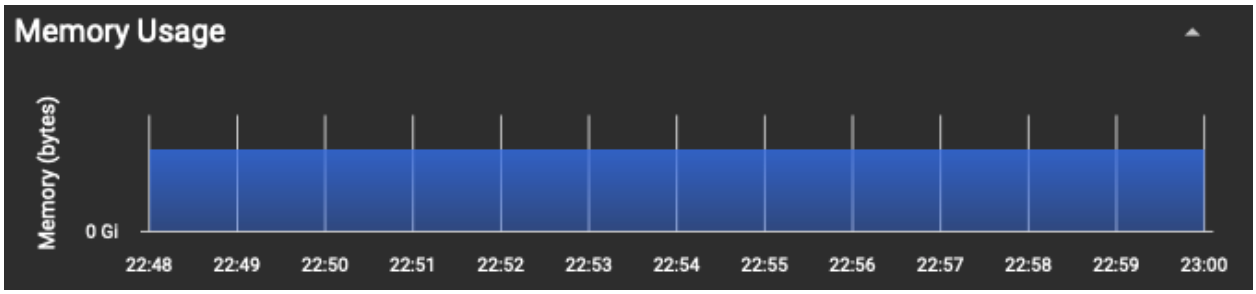

Рисунок 1.4 – Скриншот графика загрузки памяти сервиса с camunda

Данная утилита не оказывает значимое влияние на загрузку системы, поэтому её можно безопасно использовать для сбора и аналитики данных.

1.1.10 Использованные технологии

При реализации были использованы следующие технологии и библиотеки:

1. Java 17 — язык реализации.
2. Spring Boot 2.7 — фреймворк для создания REST-контроллера и внедрения зависимостей.
3. Camunda 7 Java Client — для вызова исторического API.
4. Apache POI (XSSFWorkbook) — для генерации Excel-файлов.
5. Jackson — для парсинга JSON-файла с человекочитаемыми именами активностей.
6. Swagger — для документирования эндпоинта.
7. SLF4J — для логирования.

1.1.11 Почему были выбраны технологии

Некоторые технологии уже использовались на проекте, поэтому было решено взять из для избежания добавления новых зависимостей в проект. Технологии которые уже были использованы – протестированы на продуктивных стендах.

- Apache POI выбран, так как он является стандартом для работы с Excel в Java, поддерживает потоковую запись больших файлов.
- Camunda 7 Java Client использовался вместо прямых HTTP-вызовов, так как клиент поддерживается официальным сообществом инструмента.
- Jackson взят как стандартный инструмент Spring Boot — не требует дополнительных зависимостей.
- Swagger — является достаточно легковестным расширением OpenAPI для документирования кода.

1.1.12 Вывод

Сделан сервис, который реализует API для получения истории событий Camunda за произвольный временной диапазон с параметрами startTime и endTime, передаваемыми через URI. Согласно требованиям – возвращаемые данные включают активности типов: service task, user task, exclusive gateway, send task, start event, script task, inclusive gateway, intermediate message throw event. Сделан запрашиваемый формат выгрузки — .xlsx, со столбцами Activity ID, Activity Name, Start Time, End Time, Duration (ms), Activity Type. После разработки был проведён замер, который показал, что сбор истории выполняется без заметного влияния на производительность основной системы и не загружает процессор более чем на 5Ошибки при запросе возвращают корректный HTTP-ответ с кодами: 400 (некорректные параметры), 500 (внутренняя ошибка).

1.2 Дописывание получения движений токена по workflow

1.2.1 Задача

- Получение истории и статуса токенов переведено на асинхронную схему: Camunda самостоятельно публикует уведомления о событиях в брокер сообщений.
- На стороне бизнес-логики реализован модуль подписки на эти уведомления без периодического опроса (polling).
- Подключение брокера выполнено в виде плагина Camunda Engine, активируемого при старте standalone-сборки.
- Система корректно восстанавливает состояние подписки после перезапуска.

1.2.2 Аналитика задачи

Задача была затрона на обсуждении командных задачек 05.03.2026. Изначально эта часть системы была сделана с минимальными временными затратами, поэтому легкость решения влияла на загрузку системы прямолинейно. В начале в имеющийся механизм меня начал погружать руководитель

группы, так как нужно было дать вводный контекст по коду. Основная проблема заключалась в том, что отсутвовал событийный механизм оповещения передвижения токена по бизнес процессу. Поэтому самым быстрый вариантом было опрашивать каждый заданный период времени камунду, искать по истории место токена. Для быстрого отклика системы период времени был сделан очень маленьким – 0.5 секунды. Действия, которые происходили каждые 0.5 секунд, были достаточно трудоёмкие и излишне перегружали системы. Даже при таком маленьком инервале опроса, реактивность оповещения пользователя иногда отставала. Во время обсуждения текущей ситуации с нами присутствовал системный аналитик, который параллельно формулировал задачу для разрабоки. Далее задача была передана для согласования архитектору, который перевёл её далее на меня, а я перевёл её в статус ”В процессе”. Архитектор добавил в задачу пожелание: убрать запись исторических записей в основную базу данных. Сама история оказалась бесполезной для камунды, она нужна только для отслеживания процесса в бизнес логике или аудита событий.

1.2.3 Исследование точек внедрения

После того, как я взял задачу в выполнение, сразу решил изучить внутренние механизмы основного модуля камунды, который и служит центровой точкой для любых манипуляций с процессом. По коду и документации [3] стало видно, что имеет смысл взять $HistoryEventHandler$, который является абстрактным классом и предоставляет сигнатуры нужных нам методов. Архитектура будет выстроена следующим образом – над основным сервисом $Camunda$ будет подключаться дополнительное расширение, которое будет содержать сам $Handler$, который будет отвечать за пересылку сообщений от общей точки получения сообщений. Сама система уведомлений в двигателе $camunda$ устроена следующим образом: всем объектам, которые реализуют интерфейс HistoryEventHandler и переопределяют его методы, будет подаваться сигнал о сообщении и само сообщение в метод $handleEvent/handleEvents$. Далее в плагине я решил предусмотреть так же некую обработку сообщений перед пересылкой её в kafka. Обработка заключается в упаковке полученной сущности в объект для транспортировки. Далее сообщение будет передаваться в топик $kafka$ и после этого

на стороне бизнес логики будет обработчик сообщений от $kafka$, который будет брать сообщение и далее обрабатывать. На уровне бизнес логики обработка сообщений будет сводиться к обработке трёх видов событий. Так и будет происходить синхронизация процесса из $Camunda$ с состоянием того же процесса в бизнес логике. События будут следующего вида: процесс завершился/отменился, тогда мы должны изменить его статус в бизнес логике; задача в процессе создана/удалена, тогда создаём/закрываем шаг; активность сменилась, тогда двигаем lifecycle-состояние. При этом нужно предусмотреть механизм повторной обработки при ошибке.

1.2.4 Handler

Я использовал достаточно простую идею для написания этого сервиса на сторона бизнес логики: приходит событие из $kafka$, я его забираю и отражаю в нашей модели. Далее как раз логика разделяется на 3 вида событий.

Начал я с метода $consumeProcessInstanceEvent$ — самый простой из трёх. Ищу объект по $rootProcessInstanceId$. Когда нашёл — смотрю в каком состоянии завершился процесс: если «прерван» или «прерван извне» — ставлю «отменён». Если «завершён» — всё прошло по сценарию и тогда можно поставить «финиш». Заодно, пока уже есть объект в быстром доступе, фиксирую дату завершения, если она ещё не проставлена. После этого отдаю в сервис $workflowObjectSubscriberBean$ в метод $workflowFinished$, которые ранее были уже реализованы, так как наша задача не переписать с нуля, а внедрить новый механизм. Следующий случай — когда объект вовсе не нашёлся. Это меня попросили считать не ошибкой: событие просто прилетело раньше, чем объект успел создаться, такое может быть из-за задержек сети и других накладных расходов. Таких гонок не избежать, поэтому я просто возвращаю то же событие обратно в очередь с задержкой в 100 миллисекунд. Оно обождёт и попробует снова.

Следующий вариант события я выделил в $consumeTaskInstanceEvent$ — этот случай немного сложнее. Здесь мне достаточно базового объекта ($BaseWorkflowObject$), без лишней загрузки сторонних данных из медленной персистенции. На «удалить» и «завершить» логика одинакова — задача закрыта в контексте процесса, замечу это не значит что закрыт процесс, передаю в $processFinishedTask$, который был ре-

ализован ранее и у нас опять получилось встроиться в текущий механизм без его переписывания. На «создать» — другой случай. Тут уже нужен полный объект с данными из персистенции, поэтому сначала вызываю $createWorkflowObjectFromNuxeo$, и только потом иду в $processCreateTaskInstanceEvent$. Там по $taskDefinitionKey$ нахожу шаблон шага и передаю всё в $processIncomingTask$. Был старый путь, который откладывал обработку в зависимости от текущего статуса объекта. Я его закомментировал: сейчас логика упрощена, всегда идём напрямую, но нужно сохранить логику старой реализации до окончания тестирования.

И последний сценарий ($consumeActivityInstanceEvent$) — отвечает за переходы жизненного цикла. Здесь я внедрил фильтр: меня интересует только тип $intermediateMessageCatch$. Остальные типы — шлюзы, подпроцессы, служебные задачи — тоже генерируют события, но они мне не нужны. Дальше собираю строку состояния: беру префикс по типу активности и прибавляю $activityId$. Если в идентификаторе есть символ # — обрезаю всё после него, это служебный суффикс Камунды, в моей модели он не нужен. Важно проверить перед тем как что-то менять — не стоит ли объект уже в этом состоянии и если стоит — выхожу без лишних действий, если переход разрешён жизненным циклом — достаю $LifecycleTransition$, из него беру состояние и передаю в $workflowMoveToTransient$, где старая логика сделает перевод объекта в некоторое техническое состояние, при котором действия пользователя с объектом будут невозможны. Объект движется дальше по схеме.

Код обработчика можно найти в **Приложении Б**.

1.2.5 Сервисы для обработки событий на уровне плагина

Плагин меня попросили создать в отдельном проекте и через сборку контейнера внести в плагин в другой контейнер. Данный плагин должен перехватывать события от камунды в режиме реального времени и публиковал их в шину данных Kafka, откуда любой заинтересованный сервис мог бы их получить, в частности наш перехватчик событий на уровне бизнес логики.

Прежде чем приступить к реализации, я изучил документацию Camunda BPM по механизмам расширения движка [4]. Camunda предоставляет интерфейс ProcessEnginePlugin, позволяющий вмешаться в про-

цесс инициализации движка, а также возможность добавить стандартный обработчик истории через метод setHistoryEventHandler. Как я ранее проанализировал – интерфейс HistoryEventHandler, перехватывает абсолютно все события движка централизованно, независимо от конкретного процесса. Именно этот подход и был выбран как основной.

Проект я разбил на три Maven-модуля, чтобы разделить ответственности.

Модуль $event-listener-lib$ — общая библиотека, содержащая DTO-классы событий, перечисления, конфигурацию Kafka-продюсера. Этот модуль я решил подключать как зависимость и в плагин, и в клиентскую часть. Модуль $event-listener$ — собственно плагин для Camunda, регистрируемый в движке и перехватывающий события истории. Модуль $event-listener-client$ — клиентская библиотека для любого сервиса, желающего подписаться на события Camunda, этот модуль было решено сделать заранее, так как у архитектора были ещё планы на интеграции данных событий.

Первым делом я написал DTO-классы для передачи данных о событиях. Базовый абстрактный класс BaseEvent содержит поля, общие для всех типов событий: тип события, класс события, идентификаторы корневого и текущего экземпляра процесса, ключ определения процесса и время завершения. От BaseEvent я отнаследовал три конкретных класса. $ActivityInstanceEvent$ дополнительно содержит идентификатор и тип активности. $TaskInstanceEvent$ содержит идентификатор задачи и ключ её определения. $ProcessInstanceEvent$ добавляет поле состояния процесса. Для классификации событий были введены два перечисления: $EventClass$ – `PROCESS_INSTANCE`, `ACTIVITY_INSTANCE` или `TASK_INSTANCE` и $EventType$, который определяет, что именно произошло: `START`, `END`, `CREATE`, `DELETE`, `COMPLETE`. Аналогичным образом я написал $ProcessInstanceState$ — перечисление всех возможных состояний процесса. Параметры подключения к брокеру и название топика вынесены в переменные окружения через вспомогательный класс $SystemEnvironmentUtil$. Это позволяет деплоить плагин в разных окружениях без пересборки. Так же для конфига я предусмотрел стандартные значения, чтобы плагин мог запускаться без явного указания конфигурации. Конфигурация $Kafka$-продюсера регистрирует фабрику, шаблон и

сериализатор. Я ввёл собственное поле $eventClass$, по которому консюмер самостоятельно определяет нужный конкретный тип при десериализации. Сам продюсер EventProducer принимает $BaseEvent$ и отправляет его в настроенный топик.

После этого я начал писать главный модуль – event-listener. Точка входа плагина — класс $EventListenerPlugin$, реализующий интерфейс $ProcessEnginePlugin$. В методе вызываемом Camunda до полного запуска движка, поднимается собственный контекст плагина, который и помог мне быстро переиспользовать все инструменты $Spring - boot$. После запуска контекста из него извлекается бин $HistoryEventHandler$ и устанавливается в конфигурацию движка. Класс EventHandlerKafka реализует интерфейс HistoryEventHandler, как раз он и является получателем всех событий истории от движка. Его логику я специально сделал минималистично – передать на преобразование и, если результат есть, отправить в Kafka. Метод $handleEvents$ делегирует $handleEvent$, чтобы не дублировать логику. Вся работа по фильтрации и маппингу сосредоточена в классе $EventHandlerManager$. Изначально я забыл вписать фильтрацию процессов и демонстрационные процессы засоряли топик служебными событиями, но далее был дописан фильтр. Далее я проверяю тип события и создаю объект для пересылки. Логика фильтрации по типам событий строилась постепенно — в ходе тестирования я несколько раз запускал тестовые процессы и анализировал поток событий, определяя, какие из них несут реальную смысловую нагрузку: для $ActivityInstance$ публикуется только событие $start$, и только если активность не является userTask — задачи передаются отдельным каналом через $TaskInstance$; для $TaskInstance$ публикуются события $create$, $complete$ и $delete$; для $ProcessInstance$ отсекаются служебные события $update$, $create$, $start$, $migrate$, а публикуются лишь события завершения и отмены процесса.

Клиентская библиотека лежит в отдельном модуле $event - listener - client$. В ходе обсуждения с архитектором было решено реализовать не просто $Kafka$-консюмер, а готовую библиотеку, которая реализует интерфейс и получает уже типизированные объекты событий, не занимаясь самостоятельно десериализацией.

Интерфейс $EventListenerHandler$ определяет три метода. Каждый

метод возвращает объект будущего, что даст потребителю возможность обрабатывать события асинхронно. $BaseEvent$ является абстрактным классом, стандартный $Jackson$-десериализатор не может самостоятельно определить, в какой конкретный тип преобразовать JSON, поэтому я реализовал $BaseEventDeserializer$.

Конфигурация консюмера KafkaConsumerConfig намеренно выставляет concurrency = 1 и ручное подтверждение offset через режим `MANUAL_IMMEDIATE`. Это я сделал чтобы обеспечить упорядоченную обработку событий. В ходе промежуточных тестов я столкнулся с проблемой – один бизнес-процесс может порождать несколько событий практически одновременно, и если обрабатывать их в произвольном порядке, возникает риск гонки состояний, например, на что я и наткнулся – событие завершения задачи обрабатывалось раньше события её создания. Разработать решение этой проблемы мне помог старший разработчик, предложивший использовать очереди на уровне rootProcessInstanceId. Класс $CamundaEventProcessMap$ хранит словарь, где ключ — это $rootProcessInstanceId$, а значение — очередь событий для данного процессного экземпляра. При появлении нового события оно добавляется в очередь своего процесса. Класс $CamundaEventProcessDeque$ представляет очередь событий одного процессного экземпляра. При создании он сразу запускает асинхронный цикл обработки: поочерёдно извлекает события из очереди и передаёт их в процессор, дожидаясь завершения каждого через .join(). Консюмер $EventListenerConsumer$ подписывается на топик, при получении каждого сообщения помещает событие в $CamundaEventProcessMap$ и сразу подтверждает offset. Все запронутые классы и конфигурация прикреплены в `Приложении Б`.

1.2.6 Затронутое API

В процессе разработки плагина я успел поработать с несколькими уровнями API Camunda BPM.

Основной точкой расширения стал интерфейс $ProcessEnginePlugin$ из пакета org.camunda.bpm.engine.impl.cfg. Он предоставляет три метода жизненного цикла: $preInit$, $postInit$ и $postProcessEngineBuild$. Для целей плагина был использован только $preInit$. Именно через метод

$setHistoryEventHandler$ производится подмена стандартного обработчика.

Интерфейс $HistoryEventHandler$ из пакета org.camunda.bpm.engine .impl.history.handler определяет два метода: handleEvent(HistoryEvent) для обработки одиночного события и handleEvents(List<HistoryEvent>) для пакетной обработки. Как раз это достаточно центровое место для интеграции.

Для работы с конкретными типами исторических событий использовались три класса-сущности из пакета org.camunda.bpm.engine.impl.history.event: событие элемента активности процесса; событие пользовательской задачи; событие экземпляра процесса.

Все они наследуются от базового класса $HistoryEvent$, который предоставляет общие поля: тип события, $processDefinitionKey$, $processInstanceId$, $rootProcessInstanceId$. Конкретные классы добавляют специфические поля — например $taskId$ и $taskDefinitionKey$, $state$ и $endTime$.

Поля $eventType$ и $processDefinitionKey$, оказалось могут быть пустыми при определённых внутренних событиях движка, поэтому я добавлял валидацию любых входных значений перед любой обработкой.

1.2.7 Дописывание API

Помимо использования API Camunda, в рамках проекта я сформировал API клиентской библиотеки $event - listener - client$, предназначенный для сервисов-потребителей.

Интерфейс $EventListenerHandler$ я намеренно сделал максимально простым: три метода, по одному на каждый класс событий, каждый принимает типизированный объект и возвращает объект для асинхронной обработки.

Для подключения клиентской библиотеки к проекту-потребителю достаточно добавить зависимость $event - listener - client$ в $pom.xml$ и реализовать интерфейс $EventListenerHandler$. Библиотека автоматически настраивает консюмер, десериализацию и механизм упорядоченной обработки.

Параметры подключения к Kafka задаются через переменную окружения. Если переменная не задана, библиотека использует значение по умолчанию kafka:9876.

1.2.8 Результаты сообщений из брокера

После запуска Camunda с подключённым плагином в топик camunda-events начинают поступать сообщения — по одному на каждый класс событий. Все сообщения сериализуются в JSON.

Сообщение о старте активности (например, сервисной задачи или шлюза) имеет вид:

Listing 4: Сообщение о старте активности

```
{
  "eventType": "START",
  "eventClass": "ACTIVITY_INSTANCE",
  "rootProcessInstanceId": "523a0ff1-9c5e-404d-8f98-7a02096a29d7
   ",
  "processInstanceId": "313fab8c-606a-4abd-a3f1-a6c7b027a3d7",
  "processDefinitionKey": "kigimi-process",
  "endTime": null,
  "activityId": "serviceTask_312d1",
  "activityType": "serviceTask"
}
```

Сообщение о событии пользовательской задачи (создание, завершение или удаление):

Listing 5: Сообщение о событии пользовательской задачи

```
{
  "eventType": "CREATE",
  "eventClass": "TASK_INSTANCE",
  "rootProcessInstanceId": "57271022-1de0-4623-8c01-660ae3e743b3
   ",
  "processInstanceId": "dbde1229-5675-4745-97fa-61241c2e9b8e",
  "processDefinitionKey": "subfolder-lock",
  "endTime": null,
  "taskId": "userTask_32das3",
  "taskDefinitionKey": "userTask_approve"
}
```

Сообщение о завершении процесса:

Listing 6: Сообщение о завершении процесса

```
{
  "eventType": "END",
```

```
  "eventClass": "PROCESS_INSTANCE",
  "rootProcessInstanceId": "d2fa2c69-4028-46c3-bc7f-191ef75cb895
   ",
  "processInstanceId": "69383101-2a7c-4da6-a44a-cf42fcaecc05",
  "processDefinitionKey": "folder-first-pop-work",
  "endTime": "2025-03-09T10:42:17.000+00:00",
  "state": "COMPLETED"
}
```

Поле $eventClass$ присутствует в каждом сообщении и является ключевым для десериализации. Поле $rootProcessInstanceId$ позволяет связывать события дочерних вызванных подпроцессов с корневым процессом — это поле используется как ключ в $CamundaEventProcessMap$ для упорядоченной обработки.

1.2.9 Проверка корректного восстановления

Для более прозрачной проверки в $EventListenerConsumer$ я добавил лог, полная реализация есть в Приложении Б:

Listing 7: Логи при падении сервиса

```
@Service
  @RequiredArgsConstructor
  public class EventListenerConsumer {

    private static final Logger LOG =
  LoggerFactory.getLogger(EventListenerConsumer.class);

    private final EventListenerHandler eventListenerHandler;
    private final CamundaEventProcessMap map;

    @KafkaListener(topics = KafkaConsumerConfig.EVENT_TOPIC_NAME,
        containerFactory = "baseEventListenerContainerFactory")
    public void consumeEvent(BaseEvent event, Acknowledgment
  acknowledgment) {
      LOG.info("Received event: {}", event);
      map.put(event, e -> {
        ...
      });
      acknowledgment.acknowledge();
    }
  }
```

Для воспроизведения сценария восстановления после перезапуска я попросил развернуть девопсера на тестовом стенде: Camunda с подключённым плагином, Kafka и сервис бизнес логкии с потребителем с реализацией $EventListenerHandler$. В Camunda был запущен тестовый процесс с несколькими последовательными пользовательскими задачами. Далее намеренно завершил сервис с потребителем в момент обработки событий. Для этого я отправил $kill - 9$ процессу. После этого я перезапустил сервис. При завершении процесса по логике offset не был закоммичен, поэтому при следующем старте консюмер начал читать сообщения с последней подтверждённой позиции. Ниже приведены логи, полученные в ходе воспроизведения:

Listing 8: Логи при падении сервиса

```
2026-03-24 11:23:41.512  INFO 18432 --- [ntainer#0-0-C-1]
o.s.k.l.ConcurrentMessageListenerContainer :
   baseEventListenerContainerFactory:
stopping
2026-03-24 11:23:41.513  INFO 18432 --- [ntainer#0-0-C-1]
o.a.k.clients.consumer.KafkaConsumer      : [Consumer
clientId=consumer-rabis-group-1, groupId=rabis-group] Revoke
   previously assigned
partitions [camunda-events-0]
2026-03-24 11:23:41.514  INFO 18432 --- [ntainer#0-0-C-1]
o.s.k.l.ConcurrentMessageListenerContainer :
   baseEventListenerContainerFactory:
stopped normally, last committed offset: {camunda-events-0=14}
```

В данном случае последний закоммиченный offset равен 14, но я следил за топиком и там было на момент падения уже больше сообщений, что как раз и является не подтвержённым смещением.

Listing 9: Логи при перезапуске сервиса

```
 2026-03-24 11:24:05.301  INFO 19104 --- [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer   : Tomcat initialized
   with port(s): 8080
  (http)
2026-03-24 11:24:05.744  INFO 19104 --- [ntainer#0-0-C-1]
o.s.k.l.ConcurrentMessageListenerContainer :
```

```
   baseEventListenerContainerFactory:
partitions assigned: [camunda-events-0]
2026-03-24 11:24:05.745  INFO 19104 --- [ntainer#0-0-C-1]
o.a.k.clients.consumer.KafkaConsumer      : [Consumer
clientId=consumer-rabis-group-1, groupId=rabis-group] Setting
   offset for
partition camunda-events-0 to the committed offset FetchPosition{
   offset=14,
offsetEpoch=Optional.empty,
currentLeader=LeaderAndEpoch{leader=Optional[kafka:9092], epoch=0
   }}
2026-03-24 11:24:05.801  INFO 19104 --- [ntainer#0-0-C-1]
r.r.c.e.client.EventListenerConsumer      : Received event:
TaskInstanceEvent(eventType=CREATE, eventClass=TASK_INSTANCE,
rootProcessInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processDefinitionKey=contract-approval, endTime=null,
taskId=4d9e2b13-d947-11ef-b17e-0242ac130003, taskDefinitionKey=
   task_legal_review)
2026-03-24 11:24:05.803  INFO 19104 --- [ntainer#0-0-C-1]
r.r.c.e.client.EventListenerConsumer      : Received event:
TaskInstanceEvent(eventType=COMPLETE, eventClass=TASK_INSTANCE,
rootProcessInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processDefinitionKey=contract-approval, endTime=null,
taskId=4d9e2b13-d947-11ef-b17e-0242ac130003, taskDefinitionKey=
   task_legal_review)
2026-03-24 11:24:05.805  INFO 19104 --- [ntainer#0-0-C-1]
r.r.c.e.client.EventListenerConsumer      : Received event:
ProcessInstanceEvent(eventType=END, eventClass=PROCESS_INSTANCE,
rootProcessInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processInstanceId=3c8f1a02-d947-11ef-b17e-0242ac130003,
processDefinitionKey=contract-approval, endTime=2026-03-24T11
   :23:40.991+00:00,
state=COMPLETED)
```

Ожидаемое поведение: при жёстком завершении сервиса и незакоммиченных offset брокер повторно доставляет пропущенные сообщения после перезапуска, а механизм $CamundaEventProcessDeque$ обеспечивает их обработку в корректном порядке.

1.2.10 Вывод

В ходе задачи была внедрена новая подсистема, которая основывается на событийном подходе, который в свою очередь призван убрать опрос персистенции. Поэтому получении истории и статуса токенов переведено на асинхронную схему, $Camunda$ самостоятельно публикует уведомления о событиях в брокер сообщений. На стороне бизнес-логики реализован обработчик подписки на эти уведомления без периодического опроса. Подключение брокера выполнено в виде плагина Camunda Engine, активируемого при старте standalone-сборки. Система корректно восстанавливает состояние подписки после перезапуска.

1.3 Настройка движка Camunda под нагрузку

1.3.1 Задача

- Настроен встроенный брокер Camunda для эффективной работы при высокой нагрузке с external tasks.
- Job Executors оптимизированы под асинхронное выполнение задач с минимальным временем ожидания.
- Запись истории в слой персистенции отключена; при этом логика работы брокера полностью сохраняется.

1.3.2 Аналитика задачи

Потребоность в задаче появилась после нагрузочных тестов, которые выявили некоторые промедления при взаимодейтсвии с системой бизнес процессов. 25.03.2026 после окончательного аудита и профилирования сервисов, задача получила высший приоритет, поэтому я решил её взять, чтобы научиться конфигурировать готовые системы под тяжёлую нагрузку. Для постановки задачи мы собрались с системным аналитиком и архитектором. Обнаружилось, что камунда при реализации шаблона $externaltasks$, использует свой встроенный брокер сообщений для делегирования исполнения кода на сторону бизнес логики. Я заметил, что есть некоторые задержки при исполнении сервисных задач, при этом задержка происходит до исполнения задачи на стороне бизнес логики, что являлось показателем больших таймаутов на опрос топиков с задачами. Для более быстрой обработки нужно

найти и отрегулировать конфигурацию движка Camunda. Далее само выполнение и забор сервисных задач выполнялся без многопоточной обработки, так следует из документации [5]. Аналитик и архитектор также подметили, что для разгрузки слоя персистенции можно отключить запись истории в таблицы $postgres$, так как это не входит в перечень обязательного аудита, а сами исторические таблицы не нужны для работы процессов, так как есть операционные таблицы, в которых хранятся динамические данные процесса, поэтому было решено изначально выключить историю, а в будущем, перенаправить её в отдельную базу данных.

1.3.3 Написание конфигурации Job Executor

Как я вычитал из документации – по умолчанию движок запускает $JobExecutor$ с минимальными параметрами, ориентированными на однопоточное выполнение.

Для настройки я открыл файл `bpn-platform.xml`, который копируется в контейнер и применяется при запуске движка Camunda. В этом файле я добавил следующие параметры: параметр $core-pool-size$ задаёт базовое число потоков в пуле – я установил значение 3, чтобы движок постоянно держал несколько потоков готовыми к захвату задач; параметр $max-pool-size$ ограничивает максимальный рост пула при пиковой нагрузке – выставлено значение 10, которое пропорционально потокам процессора, которые выделены в рамках лимита $K8S$ для пода Camunda; параметр $max-jobs-per-acquisition$ определяет, сколько задач движок захватывает за один цикл опроса; я сделал увеличение с 1 до 10, что позволит обрабатывать пачки задач за один обход без многократных обращений к брокеру; Для снижения задержек так же оказался полезен параметр $wait-time-in-millis$, так как уменьшение интервала ожидания между циклами опроса до 300 мс позволило существенно сократить время от появления задачи в очереди до её захвата воркером. Параметр $lock-time-in-millis$ оставлен достаточно большим (300000 мс), чтобы задача не освобождалась повторно во время длительной обработки.

Полный листинг конфигурации вынесен в приложение Г.

1.3.4 Настройка параметров длительного опроса

Клиенты внешних задач периодически опрашивают REST API движка на наличие новых заданий по своим топикам. При коротком тайм-ауте клиент получает пустой ответ и в нашей системе, так как реализована система повторов, то вынужден тут же отправлять следующий запрос, создавая избыточную нагрузку на брокер.

Для устранения этого я настроил механизм длительного опроса: установил параметр $async - response - timeout$ равным 10000мс. При данной настройке движок удерживает соединение до появления задачи или до истечения тайм-аута, после чего возвращает ответ. После того, как я удостоверился при помощи инструментального профилирования с замерами времени выполнения методов и сбором сквозной истории, что клиенты начали получать задачи быстрее. Далее я выставил параметр lock-duration в 20000мс.

1.3.5 Отключение уровня хранения истории

Camunda поддерживает несколько уровней хранения истории: full, audit, activity, none. Как я убедился, по умолчанию уровень установлен в full, при котором движок записывает в исторические таблицы персистенцию — переходы по шагам, значения переменных, временные метки. Эти записи выполняются синхронно в рамках той же транзакции, что и продвижение токена. Как раз в этом случае мы и можем стать узким местом, или ещё что хуже – перегружать базу историей.

Поскольку исторические таблицы не используются ни для мониторинга, ни для аудита в рамках текущего контура, я установил параметр none. При данном значении движок полностью пропускает запись в исторические таблицы, не затрагивая операционные таблицы (`ACT_RU_*`).

1.3.6 Конфигурационные файлы

Итоговые конфигурационные файлы, применённые в рамках данной задачи, вынесены в приложение Г. Данный конфиг подключается после сборки контейнера с плагинами, он передаётся в контейнер с образом camunda, чтобы при старте она самогла применить не стандартные настройки.

1.3.7 Проверка быстрой асинхронной работы job executors

Для проверки корректности настройки Job Executor я запустил несколько экземпляров процессов, содержащих сервисные задачи, которые как раз реализуют external tasks. Во время профилирования было заметно, как задачи стали захватываться именно пакетами и распределяться по нескольким потокам из пула.

1.3.8 Проверка отсутствия истории в слое персистенции при движении по схеме

После отключения истории я запустил несколько экземпляров процессов (из проверки предыдущих шагов) и выполнил прямые запросы к базе данных PostgreSQL на исторические таблицы `ACT_HI_PROCINST`, `ACT_HI_ACTINST` и `ACT_HI_TASKINST`. Во всех таблицах отсутствовали новые записи, соответствующие запущенным экземплярам, что подтвердило отключение записи истории. Одновременно я проверил также динамические таблицы: там , как я увидел были всё те же операционные данные активных экземпляров. Процессы создавались без проблем и жили без аварийных завершений

1.3.9 Вывод

В рамках задачи я настроил движок Camunda для работы с высоконагруженной системой. Настроил Job Executor за счёт увеличения пула потоков и сокращения интервала опроса устранила задержки при захвате сервисных задач. Включил механизм длительного опроса для клиентов external tasks, что снизило число избыточных запросов к брокеру. Отключил уровень хранения истории и тем самым разгрузил слой персистенции без нарушения логики исполнения процессов.

1.4 Написание кеша для метаданных

1.4.1 Задача

- Реализован read model кеш, наполняемый при старте сервиса из слоя персистенции.
- Кеш подключается как отдельная библиотека с минимальным на-

бором зависимостей.

– Инвалидация кеша осуществляется по событиям через event listener без перезапуска сервиса.

– При обращении к данным из кеша количество обращений к базе данных происходит только на заполнение кеша.

– Кеш корректно обновляется при получении события изменения метаданных.

1.4.2 Аналитика задачи

Данная задача была описана аналитиком после общих идей по введению кеширования в подсистемы. Система корректно работает только при обычной нагрузке, но при повышении нагрзуки в 10 раз, подсистемы излишне много делают запросов в однокластерную персистенцию. Две подсистемы для формульных преобразований и для трансформаций файлов так же активно ходят в персистенцию, но я заметил, что от полученных данных они используют только 30 процентов информации. Для данных двух подсистем можно реализовать некоторый общий инструмент, который позволит на старте приложения вытягивать данные из персистенции, сохранять их в оперативную память, что повысит локальность данных и скорость их получения. Данные архитектор попросил предоставить в упрощённом варианте, так как полноценные документы из документоориентированной базы будут излишне занимать место. Первичная нагрузка ожидается в 50.000 элементов по 30 полей в каждом. Поля есть как служебные, так и обычные. Read модель кеша означает то, что его принято только читать, а при изменениях он будет инвалидироваться и перезаполняться, так как изменения этой части данных будут изменяться не часто – константы. Поскольку кеш кодово будет одинаковым для обоих сервисов, то решено сделать его ввиде подключаемой библиотеки, с набором интерфейсов для кастомной реализации, если потребуется.

1.4.3 Реализация решения

Приступив к реализации, я для начала решил определить базовую структуру библиотеки. Поскольку она должна подключаться к двум неза-

висимым сервисам, я оформил её как отдельный Maven-модуль (см. Приложение Г) с зависимостями на $SpringContext$, $Jackson$ и клиент $Nuxeo$, остальное библиотека может взять после подключения к конкретному сервису, так как меня попросили её сделать максмаильно легковесной. Данная библиотекта будет иметь версию без указания $SNAPSHOT$.

1.4.4 Модель данных

После создания проекта спроектировал упрощённую модель элемента справочника — класс $DictionaryItem$ (Приложение Г). Так как архитектор обозначил требование хранить только 30% полей документа, я выделил набор служебных полей – $_id$, $_label$, $_value$, $_parent$, $_is_default$, а оставшиеся поля разместил в словаре с помощью аннотаций Jackson $JsonAnyGetter$ $JsonAnySetter$, что позволит мне не привязывать модель кеша к конкретной схеме документов базы, в которой есть около 100 излишних полей.

1.4.5 Слой получения данных

Для получения данных из персистенции я ввёл интерфейс $RdmData$ $Provider$, а его реализацию $RdmDataProviderRestImpl$ делегирует HTTP-клиенту $DataProviderRdmClient$ (Приложение Г). Клиент обращается к эндпоинту с Basic-аутентификацией и таймаутом 30 минут, так как первичная выгрузка элементов может занимать продолжительное время, а со временем ожидается только увеличение размера. Так же был написан NuxeoRdmService (Приложение Г) который позволяет получать уникальный сгенерированный номер документа для дальнейшего запроса иерарихии документов с главного узла по его ссылке из персистенции.

1.4.6 Парсинг иерархии справочников

Данные из персистенции поступают в виде вложенной JSON-структуры, так как между персистенцией и библиотекой стоит ещё один сервис, который приводит данные в транспортировочный формат.

Я определил интерфейс $RdmParserService$ и реализовал $RdmParser$ $ServiceStringImpl$ (Приложение Г). Алгоритм является рекурсивным, но далее по возможности и времени можно его разложить в линейный, он обходит JSON-дерево, ищет ключи с суффиксом который задан в $RdmClientConfig$, извлекает по ним имена справочников и разворачивает вложенные массивы

в плоскую структуру, так как иерархичность для данных сервисов не играет роли.

1.4.7 Кеш и инициализация

Главным компонентом стал сервис $RdmLibService$ (Приложение Г), он практически как оркестратор. Для хранения данных кеша я выбрал примитив $ConcurrentHashMap$ со значениями типа $CopyOnWriteArrayList$ для обеспечения безопасности при параллельных чтениях без доп блокировок. После первичного тестирования я обнаружил, что не стоит допускать пользователя до кеша, пока он не заполнился, так как возникают ошибки системы из-за пустых данных. Чтобы потребитель не мог обратиться к кешу до завершения его наполнения, я ввёл барьер на базе $CountDownLatch$ с таймаутом 10 минут:

```
private final CountDownLatch cacheInitLatch = new CountDownLatch
    (1);

public List<DictionaryItem> getDictByDocRef(String docRef) {
    cacheInitLatch.await(10, TimeUnit.MINUTES);
    ...
}
```

1.4.8 Асинхронная инициализация и повторные попытки

Заполнение кеша запускается по кастомному событию $RdmRefresh\ Event$ (Приложение Г), которое сервис-потребитель публикует при старте приложения (я сделал специально некоторое делегирование этой функции на сторону пользователя библиотеки, так как у раных сервисов понятие полного старта может отличаться). Слушатель $RefreshEventListener$ (Приложение Г) помечен аннотацией для асинхронного запуска в отдельном потоке, поэтому не блокирует поток запуска контекста. При неудаче загрузки слушатель выжидает некоторое заданное вреся и повторяет попытку – это покрывает случаи временной недоступности персистенции при старте кластера, недоступность персистенции связана с заполнением кешей документоориентированной прослойки.

1.4.9 Конфигурация

Для подключения к Nuxeo я оформил отдельный конфиг $NuxeoRdm\ Config$ (Приложение Г), читающий параметры подключения из $application\ .properties$ сервиса-потребителя. Все интерфейсы вынесены в публичное API библиотеки, чтобы при необходимости сервис мог подменить реализацию. Я сделал это для того, чтобы не менять код библиотеки, если потребуется какая-то узконаправленная функция.

1.4.10 Тестирование

Покрытие я реализовал в RdmLibServiceTest (Приложение Г), проверив три сценария: полноценный разбор вложенной структуры, разбор минимального справочника и обработку null-значений в полях. Тесты используют JSON тестовые файлы (`single_dictionary.json`, `null_dictionary.json` – Приложение Г).

1.4.11 Интеграция решения

После публикации библиотеки в корпоративный Maven-репозиторий я подключил её к обоим сервисам одинаковым образом: в `pom.xml` ввиде зависимости:

```
<dependency>
    <groupId>ru.rabis.utils.rdm</groupId>
    <artifactId>rdm-manager</artifactId>
</dependency>
```

Библиотека поставляет собственные бины, поэтому пришлось добавить в аннотации `ComponentScan` главного класса каждого приложения пакет библиотеки, чтобы контекст обнаружил и зарегистрировал их автоматически.

Для запуска наполнения кеша в каждом сервисе реализован `CommandLineRunner`, что и является отправной точкой валидного старта. После завершения старта контекста они публикуют событие `RdmRefreshEvent`, тем самым передавая управление асинхронному слушателю библиотеки, который начинает заполнение кеша.

Есть некоторый набор классов, которые являются некоторыми функциями, которые вызываются из шаблона трансформаций, каждая из них обра-

щается к кешу через `rdmLibService.getDictByDocRef(rdmName)`, минуя персистенцию.

При получении сигнала об изменении справочных данных сервис публикует повторный `RdmRefreshEvent`, что запускает инвалидацию и перезагрузку кеша. Ранее эндпоинт был уже интегрирован в сервис, но инвалидации касались немного другого кеша.

Интеграция свелась к трём шагам для каждого сервиса: добавить зависимость, расширить `@ComponentScan` и опубликовать `RdmRefreshEvent` при старте, инвалидации. Остальное — наполнение кеша, барьер готовности, повторные попытки — взяла на себя библиотека.

1.4.12 Заполенения кеша

После старта приложения `CommandLineRunner` публикует событие `RdmRefreshEvent("Init")`. Логи при успешной инициализации:

```
- Refresh event: Init
- rdm-library:1.0.1
- Vocabularies upload size: 49857
- Init success
```

Строка `Vocabularies upload size: 49857` подтверждает, что все справочники получены и помещены в `ConcurrentHashMap`. Далее защёлка снимется и у сервиса будет доступ к справочникам.

1.4.13 Чтения из кеша

В метод `getDictByDocRef` я не стал по умолчанию вписывать логгирование, так как он вызывается очень часто. Для верификации поведения я временно добавил строку логирования:

```
public List<DictionaryItem> getDictByDocRef(String docRef) {
    awaitCacheInit();
    if (docRef != null && !docRef.isBlank()) {
        CopyOnWriteArrayList<DictionaryItem> list = cache.get(
   docRef);
        log.info("Cache␣lookup␣'{}':␣{}␣items", docRef,
                 list != null ? list.size() : 0);
        return list != null ? new ArrayList<>(list) : null;
    }
    log.error("docRef␣blank");
```

```
    return null;
}
```

При чтении кеша в лог выводится:

```
Cache lookup 'Absceng': 52 items
Cache lookup 'PriceRoute': 18 items
Cache lookup 'LaborPrice': 34 items
```

1.4.14 Инвалидации кеша

При изменении справочных данных сервис публикует повторный `RdmRefreshEvent("Evict")`. Слушатель срабатывает асинхронно: в `refreshCache()` вызывается `cache.clear()`. Логи инвалидации:

```
- Refresh event: Evict
- rdm-library:1.0.1
- Vocabularies upload size: 49857
- Init success
```

Последовательность идентична первичной инициализации, как раз поэтому мы видим одинаковое кол-во загруженных элементов.

1.4.15 Снижения обращений в базу

HTTP-запрос к персистенции производится только внутри `get Hierarhy()`, который вызывается исключительно из `refreshCache()`. Все последующие обращения к справочникам обслуживались кешем. Это означает, что cache hit rate составил 100%.

1.4.16 Вывод

Реализовал read model кеш, который наполняется исключительно при старте сервиса из слоя персистенции. Логически кеш реализован ввиде библиотеки с минимальным набором зависимостей, что выделяет легковестность его зависимости. Инвалидация кеша сделаны через событийную модель через event listener без перезапуска сервиса. При обращении к данным из кеша количество обращений к базе данных происходит только на заполнение кеша.

ЗАКЛЮЧЕНИЕ

В ходе производственной, технологической практике выполнена проверка компетенций обучающегося путём работы над настоящими задачами в проекте: отработаны навыки постановки и согласования задач, работы в команде, написания и тестирования кода, внедрения нового функционала в уже имеющиеся системы, профилирования при нагрузке, сформирован отчёт о выполненных этапах.

В рамках разработки утилиты для сбора логов по событиям Camunda р реализован REST-сервис, формирующий выгрузку истории процессов с фильтрацией по временному диапазону. Проведено профилирование нагрузки, подтвердившее неинтрузивность решения. Усвоены подходы к проектированию вспомогательных сервисов, не влияющих на производительность основной системы.

В рамках доработки получения движений токена по workflow система переведена с периодического опроса на событийную модель. Реализован плагин Camunda Engine, активируемый при старте, с поддержкой корректного восстановления после перезапуска. Получен практический опыт построения event-driven архитектур.

В рамках настройки движка Camunda под нагрузку оптимизированы Job Executors для асинхронного выполнения external tasks, отключена запись истории в слой персистенции.

В рамках написания кеша для метаданных реализована библиотека, наполняемая при старте из базы данных и инвалидируемая по событиям. Подтверждено снижение числа обращений к базе данных. Усвоены принципы проектирования переиспользуемых библиотек с минимальными зависимостями и паттерн кеширования с событийной инвалидацией.

В результате практики освоены навыки работы с Camunda BPM: интеграция с интерфейсом исторических активностей, разработка плагинов для системы бизнес-процессов, настройка отдельного модуля под нагрузку. Приобретён опыт проектирования event-driven систем на базе брокера сообщений, реализации кеша с инвалидацией по событиям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Camunda 7.15. HistoricActivityInstanceApi documentation [Электронный ресурс]. – Режим доступа: URL: https://docs.camunda.org/javadoc/camunda-bpm-platform/7.15/org/openapitools/client/api/HistoricActivityInstanceApi.html (дата обращения: 16.04.2026).

2. Apache POI - Java API для работы с форматами Microsoft Office [Электронный ресурс]. – Режим доступа: URL: https://poi.apache.org (дата обращения: 16.04.2026).

3. Camunda 7.15. HistoryEventHandler documentation [Электронный ресурс]. – Режим доступа: URL: https://docs.camunda.org/javadoc/camunda-bpm-platform/7.18/org/camunda/bpm/engine/impl/history/handler/HistoryEventHandler.html (дата обращения: 05.03.2026).

4. Camunda 7.15. Interface ProcessEnginePlugin documentation [Электронный ресурс]. – Режим доступа: URL: https://docs.camunda.org/javadoc/camunda-bpm-platform/7.8/?org/camunda/bpm/engine/impl/cfg/ProcessEnginePlugin.html (дата обращения: 08.03.2026).

5. Job Executor Improvements in Camunda BPM 7.4 [Электронный ресурс]. – Режим доступа: URL: https://camunda.com/blog/2015/12/74-job-prioritization/ (дата обращения: 25.03.2026).

6. The Job Executor: What Is Going on in My Process Engine? [Электронный ресурс]. – Режим доступа: URL: https://camunda.com/blog/2019/10/job-executor-what-is-going-on-in-my-process-engine/ (дата обращения: 27.03.2026).

ПРИЛОЖЕНИЕ А
Система сбора истории камунды

Listing 10: Контроллер

```
@Slf4j
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
@Tag(name = "Activity Log API")
public class ActivityLogController {

  private final ActivityNameRegistry activityNameRegistry;

  /**
   * Скачивание журнала активностей в формате Excel.
   *
   * @param startTime Время начала периода необязательно().
   Формат: yyyy-MM-dd'T'HH:mm:ss
   * @param endTime Время окончания периода необязательно().
   Формат: yyyy-MM-dd'T'HH:mm:ss
   * @return Excelфайл- с журналом активностей
   */
  @GetMapping("/activity-log")
  @Operation(
      summary = Скачать" журнал активностей",
      description =
          Возвращает" Excelфайл- с журналом активностей Camunda
   за указанный период. "
              + Если" параметры времени не указаны, возвращается
   полный журнал.")
  @ApiResponses(
      value = {
        @ApiResponse(
            responseCode = "200",
            description = Файл" успешно сгенерирован и
   возвращен",
            content =
                @Content(
                    mediaType =
                        "application/vnd.openxmlformats-
```

```
officedocument.spreadsheetml.sheet")),
32          @ApiResponse(responseCode = "400", description =
Некорректные" параметры запроса"),
33          @ApiResponse(
34              responseCode = "404",
35              description = Нет" данных для отображения в журнале
активностей за указанный период"),
36          @ApiResponse(responseCode = "500", description =
Внутренняя" ошибка сервера")
37      })
38  public ResponseEntity<Resource> downloadActivityLog(
39      @RequestParam(required = false) @DateTimeFormat(pattern = "
yyyy-MM-dd'T'HH:mm:ss")
40          Date startTime,
41      @RequestParam(required = false) @DateTimeFormat(pattern = "
yyyy-MM-dd'T'HH:mm:ss")
42          Date endTime) {
43
44     Date currentTime = new Date();
45
46     if ((startTime != null && endTime != null && startTime.after(
endTime))
47          || (startTime != null && startTime.after(currentTime))
48          || (endTime != null && endTime.after(currentTime))) {
49      return ResponseEntity.badRequest().build();
50     }
51     try {
52
53      log.infoЗапрос(" на получение логгов по камунде");
54      File file = activityNameRegistry.generateActivityLogFile(
startTime, endTime);
55
56      if (file == null) {
57          log.warnНет(" данных для отображения в журнале
активностей");
58          return ResponseEntity.notFound().build();
59      }
60
61      InputStreamResource resource = new InputStreamResource(new
FileInputStream(file));
62
```

```
63        DateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd_HH
    -mm-ss");
64        String formattedDate = dateFormat.format(new Date());
65
66        HttpHeaders headers = new HttpHeaders();
67        headers.add(
68            HttpHeaders.CONTENT_DISPOSITION,
69            "attachment; filename=ActivityLog_" + formattedDate +
    ".xlsx");
70
71        return ResponseEntity.ok()
72            .headers(headers)
73            .contentLength(file.length())
74            .contentType(
75                MediaType.parseMediaType(
76                    "application/vnd.openxmlformats-officedocument.
    spreadsheetml.sheet"))
77            .body(resource);
78      } catch (IOException e) {
79        log.errorОшибка(" при создании файла журнала активностей",
    e);
80        return ResponseEntity.internalServerError().build();
81      }
82    }
83  }
```

Listing 11: Сервис

```
1  /**
2   * ActivityNameRegistry загружает отображения ID активностей в
     человекочитаемые имена из JSONфайла-
3   * при старте приложения.
4   */
5  @Component
6  @RequiredArgsConstructor
7  public class ActivityNameRegistry {
8    private static final Logger LOGGER = LoggerFactory.getLogger(
     ActivityNameRegistry.class);
9
10   private Map<String, ActivityInfo> activityInfoMap = new HashMap
     <>();
11
```

```
12  private final HistoricActivityInstanceApi
     historicActivityRestService;
13
14  @Value("${activity.decor.path}")
15  private String pathToActivityDecor;
16
17  /** Инициализирует отображения из JSONфайла- при старте Spring
     контейнера-. */
18  @PostConstruct
19  public void init() {
20    try {
21      ClassPathResource resource = new ClassPathResource(
     pathToActivityDecor);
22      ObjectMapper mapper = new ObjectMapper();
23      this.activityInfoMap = mapper.readValue(resource.
     getInputStream(),
24          new TypeReference<Map<String, ActivityInfo>>() {});
25      LOGGER.info("Loaded {} activity info mappings");
26    } catch (IOException e) {
27      LOGGER.error("Failed to load activity-decor.json", e);
28    }
29  }
30
31  /** Возвращает человекочитаемое имя активности по её ID. */
32  public String getName(String activityId) {
33    ActivityInfo info = this.activityInfoMap.get(activityId);
34    return info != null ? info.getName() : activityId;
35  }
36
37  /** Возвращает схему активности по её ID. */
38  public String getSchema(String activityId) {
39    ActivityInfo info = this.activityInfoMap.get(activityId);
40    return info != null ? info.getSchema() : null;
41  }
42
43  /** Класс для хранения информации об активности. */
44  @Getter
45  @Setter
46  public static class ActivityInfo {
47    private String name;
48    private String schema;
```

```
49    }
50
51    /**
52     * Генерирует активити лог.
53     *
54     * @param startTime Optional start date filter
55     * @param endTime Optional end date filter
56     * @return Generated Excel file with activity log
57     * @throws IOException If file cannot be created
58     */
59    public File generateActivityLogFile(final Date startTime, final
       Date endTime)
60        throws IOException {
61      HistoricActivityInstanceQueryDto query = new
      HistoricActivityInstanceQueryDto();
62      Optional.ofNullable(startTime).ifPresent(query::
      setStartedAfter);
63      Optional.ofNullable(endTime).ifPresent(query::
      setStartedBefore);
64
65      List<HistoricActivityInstanceDto> activities = new ArrayList
      <>();
66
67      try {
68        activities = historicActivityRestService
69            .queryHistoricActivityInstances(null, null, query).
      stream()
70            .filter(h -> h.getEndTime() != null)
71            .collect(Collectors.toList());
72
73        if (activities.isEmpty()) {
74          return null;
75        }
76
77        Workbook workbook = new XSSFWorkbook();
78        Sheet sheet = workbook.createSheet("Activity Log");
79        int rowNum = 1;
80
81        Row headerRow = sheet.createRow(0);
82        headerRow.createCell(0).setCellValue("Activity ID");
83        headerRow.createCell(1).setCellValue("Activity Name");
```

```
84      headerRow.createCell(2).setCellValue("Start Time");
85      headerRow.createCell(3).setCellValue("End Time");
86      headerRow.createCell(4).setCellValue("Duration (ms)");
87      headerRow.createCell(5).setCellValue("Process Instance ID")
    ;
88      headerRow.createCell(6).setCellValue("Schema");
89      headerRow.createCell(7).setCellValue("Parent Activity
    Instance Id");
90      headerRow.createCell(8).setCellValue("Parent Activity
    Instance Name");
91      headerRow.createCell(9).setCellValue("Called Process
    Instance Id");
92      headerRow.createCell(10).setCellValue("Root Process
    Instance Id");
93      headerRow.createCell(11).setCellValue("Parent Activity
    Instance Schema");
94      headerRow.createCell(12).setCellValue("Activity Type");
95
96
97      SimpleDateFormat timeFormat = new SimpleDateFormat("yyyy-MM
    -dd HH:mm:ss.SSS");
98
99      for (HistoricActivityInstanceDto activity : activities) {
100       Row row = sheet.createRow(rowNum++);
101       row.createCell(0).setCellValue(activity.getActivityId());
102       row.createCell(1).setCellValue(getName(activity.
    getActivityId()));
103       row.createCell(2).setCellValue(timeFormat.format(activity
    .getStartTime()));
104       row.createCell(3).setCellValue(timeFormat.format(activity
    .getEndTime()));
105       row.createCell(4).setCellValue(
106           activity.getDurationInMillis() != null ? activity.
    getDurationInMillis() : 0);
107       row.createCell(5).setCellValue(activity.
    getProcessInstanceId());
108       row.createCell(6).setCellValue(getSchema(activity.
    getActivityId()));
109       row.createCell(7).setCellValue(activity.
    getParentActivityInstanceId());
110       row.createCell(8).setCellValue(getName(
```

```
111                 activity.getParentActivityInstanceId()));
112             row.createCell(9).setCellValue(activity.
    getCalledProcessInstanceId());
113             row.createCell(10).setCellValue(activity.
    getRootProcessInstanceId());
114             row.createCell(11).setCellValue(getSchema(
115                 activity.getParentActivityInstanceId()));
116             row.createCell(12).setCellValue(activity.getActivityType
    ());
117         }
118
119         File tempFile = File.createTempFile("activityLog", ".xlsx")
    ;
120         try (FileOutputStream outputStream = new FileOutputStream(
    tempFile)) {
121             workbook.write(outputStream);
122         }
123
124         workbook.close();
125         return tempFile;
126
127       } catch (ApiException e) {
128         LOGGER.error(e.getResponseBody(), e);
129         throw new IOException("Failed to retrieve activity data", e
    );
130       }
131     }
132 }
```

ПРИЛОЖЕНИЕ Б

Дописывание получения движений токена по workflow

Listing 12: Handler

```
@Service
@RequiredArgsConstructor
public class WorkflowEventProcessor implements EventHandler {

  private static final Logger logger =
      LoggerFactory.getLogger(WorkflowEventProcessor.class);

  private final EventSender eventSender;
  private final ProcessObjectService processObjectService;
  private final ProcessObjectObserver processObjectObserver;
  private final ObjectProvider<StepUtilityService> stepUtility;

  @Override
  @Transactional
  public CompletableFuture<Void> onProcessInstance(ProcessEvent
   event) {
    return processObjectService.findByInstanceId(event.
   getRootInstanceId())
        .thenCompose(optional -> optional
            .map(obj -> {
              var holder = new AutoHolder<>(obj::getInput);
              var dirty = Holder.of(false);
              switch (obj.getStatus()) {
                case Status.FINISHED, Status.CANCELED:
                  break;
                default:
                  switch (event.getState()) {
                    case TERMINATED, EXTERNALLY_TERMINATED:
                      holder.getValue().setTransition(Status.
   CANCELED);
                      break;
                    case COMPLETED:
                      holder.getValue().setTransition(Status.
   FINISHED);
                      break;
                    default:
```

```
33                      logger.info("id={} status={}", obj.getId(),
34 obj.getStatus());
35                  }
36                  if (obj.getEndDate() == null) {
37                    holder.getValue().setEndDate(event.getEndTime
   ());
38                  }
39                  dirty.setValue(true);
40                  break;
41              }
42              if (holder.isPresent() && Boolean.TRUE.equals(dirty
   .getValue()))
43 {
44                return processObjectObserver
45                    .onFinished(new FinishedEvent(obj, holder.
   getValue(),
46                        event.getState().name()))
47                    .<Void>thenApply(r -> null);
48              }
49              return AsyncUtil.<Void>completedFuture(null);
50            })
51            .orElseGet(() -> AsyncUtil.runAsync(retry(event))));
52  }
53
54  @Override
55  @Transactional
56  public CompletableFuture<Void> onTaskInstance(TaskEvent event)
   {
57    return processObjectService.findBaseByInstanceId(event.
   getRootInstanceId())
58        .thenCompose(optional -> optional
59            .<AsyncFuture<Void>>map(base -> {
60              switch (event.getEventType()) {
61                case DELETE, COMPLETE:
62                  return stepUtility.getObject()
63                      .closeTask(base, event.getTaskId());
64                case CREATE:
65                  return processObjectService
66                      .loadFromRemote(base, Optional.of(event.
   getTaskKey()))
67                      .thenCompose(obj -> handleCreate(event, obj
```

```
  ));
68                 default:
69                   logger.warn("skip {}", event);
70                   return AsyncUtil.completedFuture(null);
71               }
72             }).orElseGet(() -> AsyncUtil.runAsync(retry(event))))
  ;
73   }
74
75   private AsyncFuture<Void> handleCreate(TaskEvent event,
  ProcessObject obj) {
76     var template = obj.getType()
77         .getStepTemplates().stream()
78         .filter(t -> t.getKey().equals(event.getTaskKey()))
79         .findAny();
80     return stepUtility.getObject()
81         .handleIncoming(event.getInstanceId(), obj,
82             event.getTaskId(), event.getTaskKey(), template)
83         .<Void>thenApply(r -> null);
84   }
85
86   @Override
87   @Transactional
88   public CompletableFuture<Void> onActivityInstance(ActivityEvent
    event) {
89     if (!accept(event)) {
90       return CompletableFuture.completedFuture(null);
91     }
92     return processObjectService.findByInstanceId(event.
  getInstanceId())
93         .thenCompose(optional -> optional
94             .map((ProcessObject obj) -> {
95               if (Objects.equals(event.getActivityId(), obj.
  getStatus())) {
96                 return AsyncUtil.<Void>completedFuture(null);
97               }
98               var state = prefix(event.getActivityType()) +
99 normalize(event.getActivityId());
100              if (state.equals(obj.getStatus())) {
101                return AsyncUtil.<Void>completedFuture(null);
102              }
```

```
103                var holder = new AutoHolder<>(obj::getInput);
104                var lifecycle = obj.getType().getLifecycle();
105                if (lifecycle.getState(obj.getStatus())
106                    .getAllowedTransitions().contains(state)) {
107                  Optional.ofNullable(lifecycle.getTransition(state
    ))
108                      .map(Transition::getDestination)
109                      .ifPresentOrElse(
110                          s -> holder.getValue().setTransition(
    state, s),
111                          () -> holder.getValue().setTransition(
    state));
112                }
113                if (holder.isPresent()) {
114                  return processObjectObserver.onPending(
115                      new PendingEvent(obj, state, ""));
116                }
117                return AsyncUtil.<Void>completedFuture(null);
118              })
119              .orElseGet(() -> AsyncUtil.runAsync(retry(event))));
120    }
121
122    private Runnable retry(BaseEvent event) {
123      return () -> {
124        try {
125          Thread.sleep(100);
126        } catch (InterruptedException e) {
127          throw new RuntimeException(e);
128        }
129        eventSender.send(event);
130      };
131    }
132
133    private boolean accept(ActivityEvent event) {
134      return switch (event.getActivityType()) {
135        case "intermediateMessageCatch" -> true;
136        default -> false;
137      };
138    }
139
140    private String prefix(String type) {
```

```
    return switch (type) {
      case "eventBasedGateway"         -> Prefixes.GATEWAY;
      case "intermediateMessageCatch"  -> Prefixes.CATCH;
      case "subProcess", "multiInstanceBody" -> Prefixes.
   SUBPROCESS;
      case "serviceTask"               -> Prefixes.SERVICE;
      default -> "";
    };
  }

  private String normalize(String id) {
    return id.contains("#") ? id.substring(0, id.indexOf("#")) :
   id;
  }
}
```

Listing 13: BaseEvent

```
@Getter
@SuperBuilder
@NoArgsConstructor
@ToString
public abstract class BaseEvent {
  private EventType eventType;
  private EventClass eventClass;
  private String rootProcessInstanceId;
  private String processInstanceId;
  private String processDefinitionKey;
  private Date endTime;
}
```

Listing 14: ActivityInstanceEvent

```
@Getter
@SuperBuilder
@NoArgsConstructor
@ToString
public class ActivityInstanceEvent extends BaseEvent {
  private String activityId;
  private String activityType;
}
```

Listing 15: TaskInstanceEvent

```
@Getter
@SuperBuilder
@NoArgsConstructor
@ToString
public class TaskInstanceEvent extends BaseEvent {
  private String taskId;
  private String taskDefinitionKey;
}
```

Listing 16: ProcessInstanceEvent

```
@Getter
@SuperBuilder
@NoArgsConstructor
@ToString
public class ProcessInstanceEvent extends BaseEvent {
  private ProcessInstanceState state;
}
```

Listing 17: EventClass

```
public enum EventClass {
  PROCESS_INSTANCE,
  ACTIVITY_INSTANCE,
  TASK_INSTANCE
}
```

Listing 18: EventType

```
public enum EventType {
  START,
  END,
  CREATE,
  DELETE,
  COMPLETE;

  public static EventType find(String name) throws Exception {
    for (EventType eventType : values()) {
      if (name.equalsIgnoreCase(eventType.name().toLowerCase()))
   {
        return eventType;
      }
    }
```

```
    throw new Exception();
  }
}
```

Listing 19: ProcessInstanceState

```

public enum ProcessInstanceState {
  ACTIVE,
  COMPLETED,
  TERMINATED,
  EXTERNALLY_TERMINATED,
  SUSPENDED,
  CANCELLED,
  NOT_DEFINED;

  public static ProcessInstanceState find(String name) throws
   Exception {
    for (ProcessInstanceState state : values()) {
      if (name.equalsIgnoreCase(state.name().toLowerCase())) {
        return state;
      }
    }
    throw new Exception();
  }
}
```

Listing 20: SystemEnvironmentUtil

```
public class SystemEnvironmentUtil {

  public static String getEnvOrDefault(String environment, String
    def) {
    var env = System.getenv(environment);
    return env != null ? env : def;
  }
}
```

Listing 21: KafkaProducerConfig

```

@Configuration
public class KafkaProducerConfig {

```

```
5   public static final String EVENT_TOPIC_NAME = "camunda-events";
6
7   private final String kafkaBrokers;
8   private final String kafkaTopic;
9
10  public KafkaProducerConfig() {
11    kafkaBrokers = SystemEnvironmentUtil.getEnvOrDefault("
     EVENT_LISTENER_KAFKA_BROKERS",
12        "kafka:9876");
13    kafkaTopic = SystemEnvironmentUtil.getEnvOrDefault("
     EVENT_LISTENER_KAFKA_TOPIC",
14        EVENT_TOPIC_NAME);
15  }
16
17  @Bean
18  public JsonSerializer<BaseEvent> eventDtoSerializer(
     ObjectMapper objectMapper) {
19    var serializer = new JsonSerializer<BaseEvent>(objectMapper);
20    serializer.setAddTypeInfo(false);
21    return serializer;
22  }
23
24  @Bean
25  public ProducerFactory<String, BaseEvent> producerFactory(
26      JsonSerializer<BaseEvent> eventDtoSerializer) {
27    return new DefaultKafkaProducerFactory<>(
28        Map.of(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
     kafkaBrokers),
29        new StringSerializer(),
30        eventDtoSerializer
31
32    );
33  }
34
35  @Bean
36  public KafkaTemplate<String, BaseEvent> kafkaTemplate(
37      ProducerFactory<String, BaseEvent> producerFactory) {
38    return new KafkaTemplate<>(producerFactory);
39  }
40
41  @Bean
```

```
  public String kafkaTopicName() {
    return kafkaTopic;
  }
}
```

Listing 22: EventProducer

```
@Service
@RequiredArgsConstructor
public class EventProducer {

  private final String kafkaTopicName;
  private final KafkaTemplate<String, BaseEvent> kafkaTemplate;

  public void send(BaseEvent eventDto) {
    kafkaTemplate.send(kafkaTopicName, eventDto);
  }
}
```

Listing 23: EventListenerPlugin

```
@Setter
@SpringBootApplication
public class EventListenerPlugin implements ProcessEnginePlugin {

  @Override
  public void preInit(final ProcessEngineConfigurationImpl
   processEngineConfiguration) {
    var application = SpringApplication.run(EventListenerPlugin.
   class);
    var eventHandler = application.getBean(HistoryEventHandler.
   class);
    processEngineConfiguration.setHistoryEventHandler(
   eventHandler);
  }

  @Override
  public void postInit(final ProcessEngineConfigurationImpl
   processEngineConfiguration) {
  }

  @Override
  public void postProcessEngineBuild(final ProcessEngine
```

```
   processEngine) {}
}
```

Listing 24: EventHandlerKafka

```
@Component
@RequiredArgsConstructor
public class EventHandlerKafka implements HistoryEventHandler {

  private final EventHandlerManager eventHandlerManager;
  private final EventProducer eventProducer;

  @Override
  public void handleEvent(final HistoryEvent event) {
    if (!eventHandlerManager.filterDefaultProcesses(event)) {
      eventHandlerManager
          .filterAndCreateHistoryEvent(event)
          .ifPresent(eventProducer::send);
    }
  }

  @Override
  public void handleEvents(final List<HistoryEvent> events) {
    events.forEach(this::handleEvent);
  }
}
```

Listing 25: EventHandlerManager

```
@Component
@RequiredArgsConstructor
@Sl4j
public class EventHandlerManager {

  public boolean filterDefaultProcesses(HistoryEvent event) {
    if (event.getProcessDefinitionKey() == null) {
      return false;
    }

    return event.getProcessDefinitionKey().equals("invoice")
        || event.getProcessDefinitionKey().equals("ReviewInvoice
   ");
  }
```

```
14
15  public Optional<BaseEvent> filterAndCreateHistoryEvent(
     HistoryEvent event)
16      throws Exception {
17    if (event.getEventType() == null) {
18      return Optional.empty();
19    }
20
21    if (event instanceof HistoricActivityInstanceEventEntity e) {
22      return buildActivityInstanceEvent(e);
23    } else if (event instanceof HistoricTaskInstanceEventEntity e
     ) {
24      return buildTaskInstanceEvent(e);
25    } else if (event instanceof
     HistoricProcessInstanceEventEntity e) {
26      return buildProcessInstanceEvent(e);
27    } else {
28      return Optional.empty();
29    }
30  }
31
32  private Optional<BaseEvent> buildActivityInstanceEvent(
33      HistoricActivityInstanceEventEntity event) throws Exception
     {
34    if (event.getActivityId() == null
35        || "userTask".equals(event.getActivityType())) {
36      return Optional.empty();
37    }
38
39    return switch (event.getEventType()) {
40      case "start" -> Optional.of(ActivityInstanceEvent.builder()
41          .eventClass(EventClass.ACTIVITY_INSTANCE)
42          .eventType(EventType.find(event.getEventType()))
43          .rootProcessInstanceId(event.getRootProcessInstanceId()
     )
44          .processInstanceId(event.getProcessInstanceId())
45          .processDefinitionKey(event.getProcessDefinitionKey())
46          .activityId(event.getActivityId())
47          .activityType(event.getActivityType())
48          .build());
49      default -> Optional.empty();
```

```
50      };
51    }
52
53    private Optional<BaseEvent> buildTaskInstanceEvent(
54        HistoricTaskInstanceEventEntity event) throws Exception {
55      return switch (event.getEventType()) {
56        case "delete", "complete", "create" -> Optional.of(
    TaskInstanceEvent.builder()
57            .eventClass(EventClass.TASK_INSTANCE)
58            .eventType(EventType.find(event.getEventType()))
59            .rootProcessInstanceId(event.getRootProcessInstanceId()
    )
60            .processInstanceId(event.getProcessInstanceId())
61            .processDefinitionKey(event.getProcessDefinitionKey())
62            .taskDefinitionKey(event.getTaskDefinitionKey())
63            .taskId(event.getTaskId())
64            .build());
65        default -> Optional.empty();
66      };
67    }
68
69    private Optional<BaseEvent> buildProcessInstanceEvent(
70        HistoricProcessInstanceEventEntity event) throws Exception
    {
71      return switch (event.getEventType()) {
72        case "update", "create", "start", "migrate" -> Optional.
    empty();
73        default -> Optional.of(ProcessInstanceEvent.builder()
74            .eventClass(EventClass.PROCESS_INSTANCE)
75            .eventType(EventType.find(event.getEventType()))
76            .state(ProcessInstanceState.find(event.getState()))
77            .rootProcessInstanceId(event.getRootProcessInstanceId()
    )
78            .processInstanceId(event.getProcessInstanceId())
79            .processDefinitionKey(event.getProcessDefinitionKey())
80            .endTime(event.getEndTime())
81            .build());
82      };
83    }
84  }
```

Listing 26: EventListenerHandler

```
public interface EventListenerHandler {

  CompletableFuture<Void> consumeActivityInstanceEvent(
   ActivityInstanceEvent event);

  CompletableFuture<Void> consumeProcessInstanceEvent(
   ProcessInstanceEvent event);

  CompletableFuture<Void> consumeTaskInstanceEvent(
   TaskInstanceEvent event);
}
```

```
@Configuration
public class KafkaConsumerConfig {


  private final String kafkaBrokers;

  public KafkaConsumerConfig() {
    kafkaBrokers = SystemEnvironmentUtil.getEnvOrDefault("
   EVENT_LISTENER_KAFKA_BROKERS",
        "kafka:9876");
  }

  @Bean
  public ConsumerFactory<String, BaseEvent> beConsumerFactory(
      BaseEventDeserializer baseEventDeserializer) {
    return new DefaultKafkaConsumerFactory<>(
        Map.of(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
   kafkaBrokers,
            ConsumerConfig.GROUP_ID_CONFIG, "group",
            "enable.auto.commit", false),
        new StringDeserializer(),
        baseEventDeserializer
    );
  }

  @Bean
  public ConcurrentKafkaListenerContainerFactory<String,
   BaseEvent>
      baseEventListenerContainerFactory(ConsumerFactory<String,
```

```
   BaseEvent> beConsumerFactory) {
27     var containerFactory = new
   ConcurrentKafkaListenerContainerFactory<String, BaseEvent>();
28     containerFactory.setConcurrency(1);
29     containerFactory.setConsumerFactory(beConsumerFactory);
30     containerFactory.getContainerProperties()
31         .setAckMode(ContainerProperties.AckMode.MANUAL_IMMEDIATE)
   ;
32     return containerFactory;
33   }
34 }
```

Listing 27: CamundaEventProcessMap

```
1  @Component
2  public class CamundaEventProcessMap {
3    private final Map<String, CamundaEventProcessDeque> map = new
     HashMap<>();
4
5    public void put(BaseEvent event, Function<BaseEvent,
     CompletableFuture<Void>> processor) {
6      synchronized (map) {
7        map.computeIfAbsent(event.getRootProcessInstanceId(),
8            id -> new CamundaEventProcessDeque(map, processor, ()
     -> map.remove(id)))
9            .add(event);
10     }
11   }
12 }
```

Listing 28: CamundaEventProcessDeque

```
1  public class CamundaEventProcessDeque {
2
3    private final Queue<BaseEvent> queue = new ArrayDeque<>();
4    private Object lock;
5    private Function<BaseEvent, CompletableFuture<Void>> processor;
6    private Runnable remove;
7
8    public CamundaEventProcessDeque(Object lock,
9        Function<BaseEvent, CompletableFuture<Void>> processor,
     Runnable remove) {
10     this.lock = lock;
```

```
    this.processor = processor;
    this.remove = remove;

    CompletableFuture.runAsync(this::processLoop);
  }
  public void add(BaseEvent event) {
    queue.add(event);
  }

  private void processLoop() {
    for (;;) {
      BaseEvent currentEvent;
      synchronized (lock) {
        currentEvent = queue.poll();
        if (Objects.isNull(currentEvent)) {
          remove.run();
          break;
        }
      }
      processor.apply(currentEvent).<Void>exceptionally(e -> {
        LOG.warn(currentEvent.toString(), e);
        return null;
      }).join();
    }
  }
}
```

Listing 29: bpm-platform.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bpm-platform xmlns="http://www.camunda.org/schema/1.0/
   BpmPlatform" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
   instance"
xsi:schemaLocation="http://www.camunda.org/schema/1.0/BpmPlatform
    http://www.camunda.org/schema/1.0/BpmPlatform ">

<process-engine name="default">
  <job-acquisition>default</job-acquisition>
  <configuration>org.camunda.bpm.engine.impl.cfg.
   StandaloneProcessEngineConfiguration</configuration>
  <datasource>java:jdbc/ProcessEngine</datasource>

```

```
  <properties>
    <property name="history">full</property>
  </properties>

  <plugins>

    <plugin>
      <class>ru.rabis.camunda.eventlistener.EventListenerPlugin</class>
    </plugin>
  </plugins>


</process-engine>

</bpm-platform>
```

Listing 30: EventListenerConsumer

```
@Service
@RequiredArgsConstructor
public class EventListenerConsumer {

  private final EventListenerHandler eventListenerHandler;
  private final CamundaEventProcessMap map;

  @KafkaListener(topics = KafkaConsumerConfig.EVENT_TOPIC_NAME,
      containerFactory = "baseEventListenerContainerFactory")
  public void consumeEvent(BaseEvent event, Acknowledgment acknowledgment) {
    map.put(event, e -> {
      if (e instanceof ActivityInstanceEvent e2) {
        return eventListenerHandler.consumeActivityInstanceEvent(e2);
      } else if (e instanceof ProcessInstanceEvent e2) {
        return eventListenerHandler.consumeProcessInstanceEvent(e2);
      } else if (e instanceof TaskInstanceEvent e2) {
        return eventListenerHandler.consumeTaskInstanceEvent(e2);
      }
      return CompletableFuture.failedFuture(new ClassCastException(e.getClass().getSimpleName()));
```

```
20        });
21        acknowledgment.acknowledge();
22    }
23 }
```

ПРИЛОЖЕНИЕ В
Конфигурация движка Camunda под нагрузку

Listing 31: bpm-platform.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bpm-platform xmlns="http://www.camunda.org/schema/1.0/BpmPlatform"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <job-executor>
    <job-acquisition name="default">
      <properties>
        <property name="maxJobsPerAcquisition">10</property>
        <property name="waitTimeInMillis">300</property>
        <property name="lockTimeInMillis">300000</property>
        <property name="corePoolSize">3</property>
        <property name="maxPoolSize">10</property>
        <property name="queueCapacity">3</property>
      </properties>
    </job-acquisition>
  </job-executor>
</bpm-platform>
```

Listing 32: bpm-platform.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bpm-platform xmlns="http://www.camunda.org/schema/1.0/BpmPlatform"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <properties>
    <property name="jobExecutorAsyncResponseTimeout">10000</property>
    <property name="jobExecutorDefaultLockTimeInMillis">20000</property>
    <property name="batchJobsPerSeed">100</property>
    <property name="history">none</property>
  </properties>
  <job-executor>
    <job-acquisition name="default">
      <properties>
```

```
13          <property name="maxJobsPerAcquisition">10</property>
14          <property name="waitTimeInMillis">300</property>
15          <property name="lockTimeInMillis">300000</property>
16          <property name="corePoolSize">3</property>
17          <property name="maxPoolSize">10</property>
18          <property name="queueCapacity">3</property>
19        </properties>
20      </job-acquisition>
21    </job-executor>
22  </bpm-platform>
```

ПРИЛОЖЕНИЕ Г

Написание кеша для метаданных

Listing 33: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="
   http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
   maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <relativePath>../pom.xml</relativePath>
  </parent>

  <packaging>jar</packaging>

  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>7.0.0</version>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>2.0.13</version>
    </dependency>
    <!-- nuxeo -->
    <dependency>
      <groupId>org.nuxeo.client</groupId>
      <artifactId>nuxeo-java-client</artifactId>
    </dependency>
    <dependency>
```

```
35        <groupId>org.projectlombok</groupId>
36        <artifactId>lombok</artifactId>
37        <scope>provided</scope>
38      </dependency>
39      <dependency>
40        <groupId>org.junit.jupiter</groupId>
41        <artifactId>junit-jupiter</artifactId>
42        <version>5.10.1</version>
43        <scope>test</scope>
44      </dependency>
45      <dependency>
46        <groupId>org.mockito</groupId>
47        <artifactId>mockito-core</artifactId>
48        <version>5.11.0</version>
49        <scope>test</scope>
50      </dependency>
51      <dependency>
52        <groupId>org.mockito</groupId>
53        <artifactId>mockito-junit-jupiter</artifactId>
54        <version>5.11.0</version>
55        <scope>test</scope>
56      </dependency>
57    </dependencies>
58
59    <repositories>
60
61    </repositories>
62
63    <build>
64      <plugins>
65        <plugin>
66          <artifactId>maven-compiler-plugin</artifactId>
67          <version>3.8.1</version>
68          <configuration>
69              <source>17</source>
70              <target>17</target>
71          </configuration>
72        </plugin>
73        <plugin>
74          <artifactId>maven-jar-plugin</artifactId>
75          <version>3.2.0</version>
```

```
      </plugin>
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.0.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Listing 34: DictionaryItem

```
public class DictionaryItem {

  private static final String IS_DEFAULT = "_is_default";
  private static final String VALUE = "_value";
  private static final String PARENT = "_parent";
  private static final String LABEL = "_label";
  private static final String ID = "_id";

  private final Map<String, Object> additionalFields;

  public DictionaryItem() {
    this.additionalFields = new HashMap<>();
  }

  public DictionaryItem(Map<String, Object> additionalFields) {
    this.additionalFields = new HashMap<>(additionalFields);
  }

  @JsonAnySetter
  public void setAdditionalField(String key, Object value) {
    additionalFields.put(key, value);
  }

  @JsonAnyGetter
  public Map<String, Object> getAdditionalFields() {
    return additionalFields;
  }

  private <T> T getField(String key, Class<T> retType) {
    Object valueByKey = additionalFields.get(key);
    if (retType.isInstance(valueByKey)) {
```

```
32        return (T) valueByKey;
33      }
34      return null;
35    }
36
37    public Boolean getIsDefault() {
38      return getField(IS_DEFAULT, Boolean.class);
39    }
40
41    public String getValue() {
42      return getField(VALUE, String.class);
43    }
44
45    public String getParent() {
46      return getField(PARENT, String.class);
47    }
48
49    public String getLabel() {
50      return getField(LABEL, String.class);
51    }
52
53    public String getId() {
54      return getField(ID, String.class);
55    }
56
57  }
```

Listing 35: RdmDataProvider

```
1  public interface RdmDataProvider {
2
3    Object getAllVocabularies(UUID rootUuid)
4        throws IOException, InterruptedException;
5
6    Object getAllVocabularies()
7        throws IOException, InterruptedException;
8  }
```

Listing 36: DataProviderRdmClient

```
1  @Slf4j
2  @Service
3  public class DataProviderRdmClient {
```

```
4
5   private static final Duration REQUEST_TIMEOUT = Duration.
     ofMinutes(30);
6   private final HttpClient httpClient = HttpClient.newBuilder()
7       .connectTimeout(REQUEST_TIMEOUT)
8       .build();
9
10  private static final ObjectMapper OBJECT_MAPPER = new
     ObjectMapper();
11
12  @Value("${url}")
13  private String url;
14  @Value("${admin.username}")
15  private String adminUsername;
16  @Value("${admin.password}")
17  private String adminPassword;
18
19  public Map<String, List<DictionaryItem>> getHierarhy()
20      throws IOException, InterruptedException {
21    URI uri = URI.create(url + "/api/v1/");
22    HttpRequest request = HttpRequest.newBuilder(uri)
23        .GET()
24        .timeout(REQUEST_TIMEOUT)
25        .header("Authorization", buildBasicAuthHeader())
26        .header("Accept", "application/json")
27        .build();
28
29    HttpResponse<String> response =
30        httpClient.send(request, HttpResponse.BodyHandlers.
     ofString(StandardCharsets.UTF_8));
31    if (isSuccess(response.statusCode())) {
32      String body = response.body();
33      return OBJECT_MAPPER.readValue(body, new TypeReference<>()
     {});
34    }
35
36    log.error("Unexpected response when requesting vocabularies:
     status={}, uri={}",
37        response.statusCode(), uri);
38    throw new IllegalStateException("Failed to fetch vocabularies
     : HTTP " + response.statusCode());
```

```
  }

  private String buildBasicAuthHeader() {
    return "Basic " + Base64.getEncoder().encodeToString(
      (adminUsername + ":" + adminPassword).getBytes(
   StandardCharsets.UTF_8));
  }

  private boolean isSuccess(int statusCode) {
    return statusCode >= 200 && statusCode < 300;
  }
}
```

Listing 37: NuxeoRdmService

```
@Slf4j
@Service
public class NuxeoRdmService {
  @Autowired
  @Qualifier("Client")
  private NuxeoClient client;

  public UUID getUuidByDocRef(String docRef) throws Exception {
    Document answerDocument = client
        .operation("GetDocument")
        .input(docRef).execute();
    if (answerDocument == null) {
      return null;
    }
    String rootUuid = answerDocument.getId();
    log.info("{} uuid:{}", RdmClientConfig.
   ROOT_DICTIONARIES_DATA_GROUP, rootUuid);
    return UUID.fromString(rootUuid);
  }
}
```

Listing 38: RdmParserServiceStringImpl

```
@Service
public class RdmParserServiceStringImpl implements
   RdmParserService {

  private final ObjectMapper objectMapper = new ObjectMapper();
```

```
5
6   @Override
7   public Map<String, List<DictionaryItem>> parseVocabularies(
     Object payload) {
8     try {
9       String strPayload = String.valueOf(payload);
10      if (strPayload == null || strPayload.isBlank()) {
11        return Map.of();
12      }
13      JsonNode root = objectMapper.readTree(strPayload);
14      Map<String, List<DictionaryItem>> result = new HashMap<>();
15      collectDictionaries(root, result);
16      return result;
17    } catch (JsonProcessingException e) {
18      throw new IllegalStateException("Failed to parse payload",
     e);
19    }
20  }
21
22  private void collectDictionaries(JsonNode node, Map<String,
     List<DictionaryItem>> accumulator) {
23    if (node == null) {
24      return;
25    }
26    if (node.isObject()) {
27      node.fields().forEachRemaining(entry -> {
28        String key = entry.getKey();
29        JsonNode value = entry.getValue();
30        if (key.endsWith(RdmClientConfig.RDM_TYPE_SUFFIX) &&
     value.isArray()) {
31          String dictionaryName = key.substring(0,
32              key.length() - RdmClientConfig.RDM_TYPE_SUFFIX.
     length());
33          if (!dictionaryName.isBlank()) {
34            for (JsonNode typeNode : value) {
35              JsonNode itemsNode = typeNode.get(dictionaryName);
36              if (itemsNode != null && itemsNode.isArray()) {
37                List<DictionaryItem> items =
38                    accumulator.computeIfAbsent(dictionaryName,
     unused -> new ArrayList<>());
39                for (JsonNode itemNode : itemsNode) {
```

```
                    items.add(toDictionaryItem(itemNode));
                  }
                }
              }
            }
          }
          collectDictionaries(value, accumulator);
        });
      } else if (node.isArray()) {
        for (JsonNode child : node) {
          collectDictionaries(child, accumulator);
        }
      }
    }

    private DictionaryItem toDictionaryItem(JsonNode itemNode) {
      Map<String, Object> additionalFields =
          objectMapper.convertValue(itemNode, new TypeReference<Map<String, Object>>() {});

      return new DictionaryItem(additionalFields);
    }
}
```

Listing 39: RdmLibService

```
@Slf4j
@Service
public class RdmLibService {

  private final RefreshService refreshService;

  private final ConcurrentHashMap<String, CopyOnWriteArrayList<DictionaryItem>> cache =
      new ConcurrentHashMap<>();
  private final CountDownLatch cacheInitLatch = new CountDownLatch(1);

  public RdmLibService(RefreshService refreshService) {
    this.refreshService = refreshService;
  }

```

```
15  public List<DictionaryItem> getDictByDocRef(String docRef) {
16    awaitCacheInit();
17    if (docRef != null && !docRef.isBlank()) {
18      CopyOnWriteArrayList<DictionaryItem> list = cache.get(
     docRef);
19      return list != null ? new ArrayList<>(list) : null;
20    }
21    log.error("docRef blank");
22    return null;
23  }
24
25  public Map<String, List<DictionaryItem>> getAllDicts() {
26    awaitCacheInit();
27
28    return cache.entrySet().stream()
29        .collect(HashMap::new,
30            (m, e) -> m.put(e.getKey(), new ArrayList<>(e.
     getValue())),
31            HashMap::putAll);
32  }
33
34  public void put(String dictionaryName, DictionaryItem
     dictionaryItem) {
35    awaitCacheInit();
36
37    String id = dictionaryItem.getId();
38    if (id == null) {
39      log.warn("DictionaryItem without _id prop");
40      return;
41    }
42
43    cache.compute(dictionaryName, (key, existingList) -> {
44      CopyOnWriteArrayList<DictionaryItem> list =
45          existingList != null
46            ? new CopyOnWriteArrayList<>(existingList) : new
     CopyOnWriteArrayList<>();
47
48      boolean found = false;
49      for (int i = 0; i < list.size(); i++) {
50        if (id.equals(list.get(i).getId())) {
51          list.set(i, dictionaryItem);
```

```
52          log.info("Update line in {} with id {}", dictionaryName
 , id);
53          found = true;
54          break;
55        }
56      }
57
58      if (!found) {
59        list.add(dictionaryItem);
60        log.info("Create new line in {} with id {}",
 dictionaryName, id);
61      }
62
63      return list;
64    });
65  }
66
67  public boolean isCacheInitialized() {
68    return cacheInitLatch.getCount() == 0;
69  }
70
71  private void awaitCacheInit() {
72    try {
73      if (!cacheInitLatch.await(10, TimeUnit.MINUTES)) {
74        log.error("Cache initialization timeout");
75        throw new IllegalStateException("Cache not initialized
 within timeout");
76      }
77    } catch (InterruptedException e) {
78      Thread.currentThread().interrupt();
79      throw new IllegalStateException("Interrupted while waiting
 for cache init", e);
80    }
81  }
82
83  public boolean refreshCache() {
84    try {
85      log.info("{}:{}", RdmClientConfig.LIB_NAME,
86          getClass().getPackage().getImplementationVersion());
87
88      Map<String, List<DictionaryItem>> vocabularies =
```

```
    refreshService.refresh();

      cache.clear();
      vocabularies.forEach((key, items) ->
          cache.put(key, new CopyOnWriteArrayList<>(items)));

      log.info("Vocabularies upload size: {}", vocabularies.size
    ());
      cacheInitLatch.countDown();
      return true;
    } catch (Exception e) {
      log.error("refreshCache failed", e);
      return false;
    }
  }
}
```

Listing 40: RdmRefreshEvent

```
@AllArgsConstructor
@Getter
public class RdmRefreshEvent {
  private final String message;
}
```

Listing 41: RefreshEventListener

```
@Component
@RequiredArgsConstructor
@Slf4j
@EnableAsync
public class RefreshEventListener {

  private final RdmLibService rdmLibService;

  @Async
  @EventListener
  public void onApplicationEvent(RdmRefreshEvent event) {
    log.info("Refresh event: {}", event.getMessage());
    while (true) {
      Boolean result = rdmLibService.refreshCache();
      if (Boolean.TRUE.equals(result)) {
        log.info("Init success");
```

```
17          break;
18        } else {
19          log.warn("Init fail (retry after 1 minute)");
20        }
21        try {
22          Thread.sleep(RdmClientConfig.DELAY_BETWEEN_ATTEMPTS);
23        } catch (InterruptedException ie) {
24          Thread.currentThread().interrupt();
25          log.error("Thread interrupted during sleep: {}", ie.
    getMessage());
26          break;
27        }
28      }
29    }
30  }
```

Listing 42: NuxeoRdmConfig

```
1  @Configuration
2  public class NuxeoRdmConfig {
3
4    @Value("${username}")
5    private String nuxeoAdminUsername;
6    @Value("${password}")
7    private String nuxeoAdminPassword;
8    @Value("${url}")
9    private String nuxeoUrl;
10
11
12   @Bean("Client")
13   public NuxeoClient nuxeoRdmClient() {
14     return new NuxeoClient.Builder()
15         .url(nuxeoUrl)
16         .authentication(nuxeoAdminUsername, nuxeoAdminPassword)
17         .schemas("*")
18         .connect();
19   }
20 }
```

Listing 43: RdmLibServiceTest

```
1  @ExtendWith(MockitoExtension.class)
2  class RdmLibServiceTest {
```

```
3
4   @Mock
5   private RefreshService refreshService;
6
7   private RdmLibService service;
8
9   @BeforeEach
10  void setUp() {
11    service = new RdmLibService(refreshService);
12  }
13
14  @Test
15  void shouldParseMinimalDictionaryStructure() throws IOException
      {
16    Map<String, List<DictionaryItem>> result = parsePayload("
     single_dictionary.json");
17
18    List<DictionaryItem> items = result.get("MinimalDict");
19    assertNotNull(items, "Minimal dictionary should be parsed");
20    assertEquals(1, items.size(), "Expected single entry");
21
22    DictionaryItem item = items.get(0);
23    assertEquals(Boolean.TRUE, item.getIsDefault());
24    assertEquals("value1", item.getValue());
25    assertEquals("parent1", item.getParent());
26    assertEquals("Bar", item.getAdditionalFields().get("Foo"));
27    assertTrue(item.getAdditionalFields().containsKey("_value"));
28  }
29
30  @Test
31  void shouldParseNullDictionaryStructure() throws IOException {
32    Map<String, List<DictionaryItem>> result = parsePayload("
     null_dictionary.json");
33
34    List<DictionaryItem> items = result.get("NullDict");
35    assertNotNull(items, "Minimal dictionary should be parsed");
36    assertEquals(1, items.size(), "Expected single entry");
37
38    assertFalse(result.containsKey("_rdm_type"), "Expected to
     ignore empty postfix key");
39    assertFalse(result.containsKey(""), "Expected to ignore empty
```

```
    key");
40    assertFalse(result.containsKey("second_rdm_type"), "Expected
   to ignore unrelated key");
41
42    DictionaryItem item = items.get(0);
43    assertEquals(null, item.getIsDefault());
44    assertEquals(null, item.getValue());
45    assertEquals(null, item.getParent());
46    assertEquals("Bar", item.getAdditionalFields().get("Foo"));
47    assertTrue(item.getAdditionalFields().containsKey("_value"));
48  }
49
50  @Test
51  void initShouldFetchVocabularies() throws Exception {
52    Map<String, List<DictionaryItem>> parsedData = parsePayload("
   single_dictionary.json");
53
54    when(refreshService.refresh()).thenReturn(parsedData);
55
56    boolean success = service.refreshCache();
57
58    assertTrue(success, "Service should complete initialization")
   ;
59    verify(refreshService).refresh();
60
61    List<DictionaryItem> cachedDictionary = service.
   getDictByDocRef("MinimalDict");
62    assertNotNull(cachedDictionary, "Cached dictionary should be
   present");
63    assertFalse(cachedDictionary.isEmpty(), "Cached dictionary
   should not be empty");
64  }
65
66  private Map<String, List<DictionaryItem>> parsePayload(String
   resourceName)
67      throws IOException {
68    RdmParserService parserService = new
   RdmParserServiceStringImpl();
69    return parserService.parseVocabularies(readPayload(
   resourceName));
70  }
```

```
71
72   private String readPayload(String resourceName) throws
     IOException {
73     ClassLoader classLoader = Thread.currentThread().
     getContextClassLoader();
74     try (InputStream inputStream = classLoader.
     getResourceAsStream(resourceName)) {
75       if (inputStream == null) {
76         throw new IOException(resourceName + " resource not found
     ");
77       }
78       byte[] bytes = inputStream.readAllBytes();
79       return new String(bytes, StandardCharsets.UTF_8);
80     }
81   }
82 }
```

Listing 44: null dictionary

```
1 {
2   "NullDict_rdm_type": [
3     {
4       "NullDict": [
5         {
6           "_value": null,
7           "_parent": null,
8           "Foo": "Bar"
9         }
10      ],
11      "": [],
12      "_rdm_type": [
13        {
14          "_value": null,
15          "_parent": null,
16          "Foo": "Bar"
17        }
18      ],
19      "second_rdm_type": "ffffffff",
20      "tr_rdm_type": [
21        null
22      ],
23      "for_rdm_type": [
```

```
24          "sadasdasdasd"
25        ]
26      }
27    ]
28  }
```

Listing 45: single dictionary

```
1  {
2    "MinimalDict_rdm_type": [
3      {
4        "MinimalDict": [
5          {
6            "_is_default": true,
7            "_value": "value1",
8            "_parent": "parent1",
9            "Foo": "Bar"
10         }
11       ]
12     }
13   ]
14 }
```